

Contents

1 字符串	2
1.1 Manacher	2
1.2 Z 函数	2
1.3 最小表示法	2
1.4 SA	2
2 数据结构	2
2.1 李超树	2
2.2 全局平衡二叉树	3
2.3 LCT	4
2.4 广义串并联图	5
2.5 二分栈	5
2.6 wqs 二分构造方案	5
2.7 O(1) RMQ	6
3 图论	7
3.1 网络流	7
3.2 费用流	7
3.3 支配树	8
3.4 一般图最大匹配	9
4 数学	10
4.1 Barret 约简	10
4.2 拉格朗日插值	10
4.3 Pollard Rho	10
4.4 min_25	11
4.5 类欧	12
5 数论	12
5.1 分数还原	12
5.2 O(V)-O(1) GCD	12
5.3 离散对数	13
5.4 中国剩余定理	13
5.5 Miller-Rabin 素性检验	13
5.6 Pollard-Rho 分解质因数	13
5.7 扩展欧几里得	14
5.8 最小模线性值	14
5.9 二次剩余	14
5.10 原根与阶	14
5.11 杜教筛	15
5.12 线性筛	15
5.13 下取整和	15

6 多项式	16
6.1 任意模数 NTT	16
6.2 多项式复合	16
6.3 多项式复合逆	18
6.4 最短线性递推	20
6.5 线性递推第 n 项	20
6.6 复数 FFT 卷积	21
6.7 多点求值	21
6.8 多项式指数	21
6.9 多项式对数	21
6.10 多项式开根	22
6.11 多项式求逆	22
6.12 带余除法	22
6.13 快速插值	22
7 计算几何	23
7.1 Delaunay 三角剖分	24
7.2 Voronoi 图	26
7.3 三维凸包	26
7.4 三维向量	27
7.5 三维平面	28
7.6 三维旋转	28
7.7 三维直线与线段	29
7.8 上凸壳	30
7.9 凸包内点判定	30
7.10 半平面交	30
7.11 圆相关求交与面积交	31
7.12 多边形包含	32
7.13 最小圆覆盖	32
7.14 最近点对	33
7.15 简单多边形三角剖分	33
7.16 多边形重心	34

1 字符串

1.1 Manacher

```
1 void manacher(int n, char *s, int *d) {
2     int l = 0, r = -1;
3     For(i, 1, n) {
4         int &k = d[i];
5         k = i > r ? 1 : min(d[l + r - i], r - i + 1);
6         while (s[i - k] == s[i + k]) k++;
7         if (i + k - 1 > r) r = i + k - 1, l = i - k + 1;
8     }
9 }
```

1.2 Z 函数

```
1 void zfunc(int n, const int *const s, int *const z) {
2     fill(z + 1, z + n + 1, 0), z[1] = n;
3     int l = 0;
4     For(i, 2, n) {
5         if (z[l] + l - 1 >= i) z[i] = min(z[i - l + 1], l + z[l] - i);
6         while (s[z[i] + 1] == s[i + z[i]]) z[i]++;
7         if (i + z[i] > l + z[l]) l = i;
8     }
9 }
```

1.3 最小表示法

```
1 int minrep(auto &a) {
2     int n = a.size(), i = 0, j = 1, k = 0;
3     while(i < n && j < n && k < n) {
4         if(a[(i + k) % n] == a[(j + k) % n]) {
5             ++k;
6         } else {
7             if(a[(i + k) % n] > a[(j + k) % n]) swap(i, j);
8             j += k + 1, j += i == j, k = 0;
9         }
10    }
11    return i;
12 }
13 // rotate(a.begin(), a.begin() + minrep(a), a.end());
```

1.4 SA

```
1 const int N = 1e6 + 10;
2 int sa[N], rk[N << 1], st[20][N];
3 void SA(int n, auto *a) {
4     static int b[N], c[N];
5     int m = *max_element(a + 1, a + n + 1);
6     For(i, 1, n) b[i] = i, rk[i] = a[i];
7     For(i, n + 1, n * 2) rk[i] = 0;
8     auto bs = [&]() {
9         memset(c, 0, m + 1 << 2);
```

```
10         For(i, 1, n) ++c[rk[i]];
11         For(i, 1, m) c[i] += c[i - 1];
12         rFor(i, n, 1) sa[c[rk[b[i]]]--] = b[i];
13     };
14     bs();
15     for (int t = 1; t < n; t += t) {
16         int tot = 0;
17         For(i, n - t + 1, n) b[++tot] = i;
18         For(i, 1, n) if(sa[i] > t) b[++tot] = sa[i] - t;
19         bs();
20         b[sa[1]] = m = 1;
21         For(i, 2, n) {
22             int x = sa[i], y = sa[i - 1];
23             b[x] = m += (rk[x] != rk[y] || rk[x + t] != rk[y + t]);
24         }
25         memcpy(rk, b, n + 1 << 2);
26         if(m == n) break;
27     }
28     int k = 0;
29     a[n + 1] = -1;
30     For(i, 1, n) {
31         k -= k > 0;
32         while (a[i + k] == a[sa[rk[i] - 1] + k]) ++k;
33         st[0][rk[i]] = k;
34     }
35     For(i, 1, __lg(n)) For(j, 1, n - (1 << i) + 1) {
36         st[i][j] = min(st[i - 1][j], st[i - 1][j + (1 << i - 1)]);
37     }
38 }
39 int lcp(int x, int y) {
40     x = rk[x], y = rk[y];
41     if(x > y) swap(x, y);
42     int k = __lg(y - x);
43     return min(st[k][x + 1], st[k][y - (1 << k) + 1]);
44 }
```

2 数据结构

2.1 李超树

```
1 namespace DS {
2     const int S = 1e7 + 10;
3     pll tr[S];
4     int ls[S], rs[S], tot;
5     ll eval(pll x, ll y) {
6         return x[0] * y + x[1];
7     }
8     void bd() {
9         memset(ls, 0, sizeof(ls)), memset(rs, 0, sizeof(rs));
10        tot = 0;
11    }
12    void ins(pll x, int &k, int l = 1, int r = 1e6) {
```

```

13     if(!k) return tr[k = ++tot] = x, void();
14     int m = l + r >> 1, fl;
15     if(eval(tr[k], m) > eval(x, m)) swap(x, tr[k]);
16     if((fl = eval(tr[k], l) <= eval(x, l)) && eval(tr[k], r) <= eval(x, r)) return;
17     !fl ? ins(x, ls[k], l, m) : ins(x, rs[k], m + 1, r);
18 }
19 ll que(int x, int k, int l = 1, int r = 1e6) {
20     if(!k) return Z;
21     int m = l + r >> 1;
22     return min(eval(tr[k], x), x <= m ? que(x, ls[k], l, m) : que(x, rs[k], m + 1,
23         r));
24 } // namespace DS

```

2.2 全局平衡二叉树

树上点权修改 + 查询最大权独立集数值, 每次修改 $O(\log n)$ ——先全局平衡化建结构, 之后点改权更新。

- 完整程序(自带 main): 输入 n, m (点权重与操作数); 每行操作 p, y , 且 p 需要先异或上一次答案(在线防作弊), 改点权后输出当前答案
- 依赖: 无(自带 2×2 max-plus 矩阵、与 $N = 10^6$ 的全局数组)
- 实现: 重链重度剖分 + 树剖点上矩乘, 矩阵 que 计算链顶的独立集
- 初始化: $dfs0$ (轻重儿子与父亲)、 $dfs1$ (矩阵的初始 f)、 $build$ (按点权重分极值点) 全部在 $main$ 内调好

坑: 输入 $p \hat{=} ans$ 的在线加密别忘了; $f[i][1]$ 初始即点权 $a[i]$; 改权用 $upd(p, y - a[p])$ 差值更新, 和 $a[p] = y$ 同步, 整棵树必须从 1 号点开始 DFS (树无向)。

```

1  const int N = 1e6 + 10;
2  const int INF = 1e9;
3  struct mat {
4      int a[2][2];
5      int *operator[](int k) { return a[k]; }
6      const int *operator[](int k) const { return a[k]; }
7      mat operator*(const mat &b) const {
8          return {
9              max(a[0][0] + b[0][0], a[0][1] + b[1][0]),
10             max(a[0][0] + b[0][1], a[0][1] + b[1][1]),
11             max(a[1][0] + b[0][0], a[1][1] + b[1][0]),
12             max(a[1][0] + b[0][1], a[1][1] + b[1][1]);
13         }
14     };
15     int n;
16     vector<int> t[N];
17     int sz[N], s[N], tp[N], dw[N], fa[N];
18     void dfs0(int u) {
19         sz[u] = 1;
20         for(auto v : t[u]) {
21             fa[v] = u;
22             t[v].erase(find(all(t[v]), u));
23             dfs0(v);

```

```

24         if(sz[v] > sz[s[u]]) s[u] = v;
25         sz[u] += sz[v];
26     }
27     if(s[u]) t[u].erase(find(all(t[u]), s[u]));
28 }
29 int dfn[N], wt[N], rt[N];
30 int f[N][2];
31 pll dfs1(int u) {
32     static int dt;
33     dfn[u] = ++dt;
34     wt[dt] = sz[u] - sz[s[u]];
35     if(!tp[u]) tp[u] = u;
36     if(!s[u]) return dw[tp[u]] = u, pll{f[u][0], f[u][1]};
37     tp[s[u]] = tp[u];
38     pll sf = dfs1(s[u]);
39     for(auto v : t[u]) {
40         auto [x, y] = dfs1(v);
41         f[u][0] += max(x, y);
42         f[u][1] += x;
43     }
44     return {f[u][0] + max(sf[0], sf[1]), f[u][1] + sf[0]};
45 }
46 struct tree_t {
47     int ls, rs;
48     mat x;
49 } tr[N << 1];
50 #define ls (tr[k].ls)
51 #define rs (tr[k].rs)
52 int build(int l, int r) {
53     if(l == r) return l << 1;
54     int m, s1 = 0, s2 = 0;
55     for(i, l, r) s1 += wt[i];
56     for(i, l, r) if((s2 += wt[i]) * 2 > s1) {
57         m = i;
58         break;
59     }
60     m -= m == r;
61     return tr[m << 1 | 1] = {build(l, m), build(m + 1, r)}, m << 1 | 1;
62 }
63 void pushup(int k) { tr[k].x = tr[rs].x * tr[ls].x; }
64 void upd(int p, mat x, int k, int l, int r) {
65     if(l == r) return tr[k].x = x, void();
66     int m = k >> 1;
67     p <= m ? upd(p, x, ls, l, m) : upd(p, x, rs, m + 1, r);
68     pushup(k);
69 }
70 mat que(int L, int R, int k, int l, int r) {
71     if(L <= l && r <= R) return tr[k].x;
72     int m = l + r >> 1;
73     if(R <= m) return que(L, R, ls, l, m);
74     if(m < L) return que(L, R, rs, m + 1, r);
75     return que(L, R, rs, m + 1, r) * que(L, R, ls, l, m);
76 }

```

```

77 #undef ls
78 #undef rs
79 void apply(int p) {
80     upd(dfn[p], {f[p][0], f[p][1], f[p][0], -INF}, rt[tp[p]], dfn[tp[p]],
        dfn[dw[tp[p]]]);
81 }
82 pll que(int p) {
83     auto res = que(dfn[p], dfn[dw[tp[p]]], rt[tp[p]], dfn[tp[p]], dfn[dw[tp[p]]]);
84     return {max(res[0][0], res[1][0]), max(res[0][1], res[1][1])};
85 }
86 void upd(int p, int dt) {
87     f[p][1] += dt;
88     auto u = [&](int k, int w) {
89         if(!k) return;
90         auto [x, y] = que(k);
91         f[fa[k]][0] += w * max(x, y);
92         f[fa[k]][1] += w * x;
93     };
94     for(; p; p = fa[tp[p]]) {
95         u(tp[p], -1);
96         apply(p);
97         u(tp[p], 1);
98     }
99 }
100 int getans() {
101     auto [x, y] = que(1);
102     return max(x, y);
103 }

```

2.3 LCT

$O(\log n)$ 摊还的动态 lct: 支持 link/cut/路径聚合(默认异或, 改 pu 可换求和或最值)。

- init(): 先 n 与 $a[N]$ 就位, 然后对 $1..n$ 初始化
- link(u, v): 连边(自动判环); cut(u, v): 断边; que(u, v): 路径聚合值; upd(u, x): 单点改值并维护
- mkr/access/splay/fd 为内部机制, 一般不用管
- 依赖: 全局 N 与点权数组 $a[N]$

坑: 若改 pu 换成加法/最值, 注意初始值(0/INF 类)与空链的传递; cut 仅当边存在时删(发现 $fa[v] == u$ 才断); 路径查询前一定要 mkr 换根。

```

1 namespace LCT {
2 #define ls(k) ch[k][0]
3 #define rs(k) ch[k][1]
4 int ch[N][2], fa[N];
5 bool lz[N];
6 int tr[N];
7 inline void pu(int k) {
8     tr[k] = tr[ls(k)] ^ tr[rs(k)] ^ a[k];
9 }
10 inline void pl(int k) {
11     if(k) lz[k] ^= 1, swap(ls(k), rs(k));
12 }

```

```

13 inline void pd(int k) {
14     if(lz[k]) pl(ls(k)), pl(rs(k)), lz[k] = 0;
15 }
16 inline bool gc(int k) { return ch[fa[k]][1] == k; }
17 inline bool ir(int k) { return ch[fa[k]][gc(k)] != k; }
18 void pda(int k) {
19     static int st[N], t;
20     for(st[++t] = k; !ir(k);) st[++t] = k = fa[k];
21     while(t) pd(st[t--]);
22 }
23 void rot(int u) {
24     int f = fa[u], &g = fa[f], t = gc(u), x = ch[u][!t];
25     if(!ir(f)) ch[g][gc(f)] = u;
26     fa[u] = g, g = u;
27     ch[u][!t] = f, ch[f][t] = x;
28     if(x) fa[x] = f;
29     pu(f);
30 }
31 void splay(int u) {
32     for(pda(u); !ir(u); rot(u))
33         if(!ir(fa[u])) {
34             rot(gc(u) == gc(fa[u]) ? fa[u] : u);
35         }
36     pu(u);
37 }
38 void access(int u0) {
39     for(int u = u0, p = 0; u; p = u, u = fa[u]) splay(u), rs(u) = p, pu(u);
40     splay(u0);
41 }
42 int fd(int u) {
43     for(access(u); ls(u); u = ls(u), pd(u)) ;
44     return splay(u), u;
45 }
46 void mkr(int u) {
47     access(u), pl(u);
48 }
49 void init() {
50     For(i, 1, n) pu(i);
51 }
52 void link(int u, int v) {
53     mkr(u);
54     if(fd(v) != u) mkr(v), fa[v] = u;
55 }
56 void cut(int u, int v) {
57     mkr(u), access(v), access(u);
58     if(fa[v] == u) fa[v] = 0;
59 }
60 void upd(int u, int x) {
61     a[u] = x, splay(u);
62 }
63 int que(int u, int v) {
64     mkr(u), access(v);
65     return tr[v];

```

```
66 }
67 } // namespace LCT
```

2.4 广义串并联图

```
1 struct SPQR_tree {
2     int m = 1, fa[N];
3     bool typ[N];
4     unordered_map<int, int> t[N];
5     bitset<N> vis;
6     int node(int u, int v, bool t) {
7         tp[+m] = t, fa[u] = t, fa[v] = t;
8         return m;
9     }
10    void add(int u, int v, int w) {
11        if(t[u].count(v)) {
12            t[u][v] = t[v][u] = node(t[u][v], w, 0);
13        } else {
14            t[u][v] = t[v][u] = w;
15        }
16    }
17    void build(int n) {
18        queue<int> q;
19        auto ins = [&](int u) { if (!vis[u] && t[u].size() <= 2) vis[u] = 1,
20        q.push(u); };
21        For(i, 1, n) ins(i);
22        while(q.size()) {
23            int u = q.front();
24            q.pop();
25            if(t[u].empty()) continue;
26            if(t[u].size() == 1) {
27                int v = t[u].begin()->first;
28                t[v].erase(u), ins(v);
29            } else {
30                int x = t[u].begin()->first, y = next(t[u].begin())->first;
31                int w = node(t[u][x], t[u][y], 1);
32                t[x].erase(u), t[y].erase(u);
33                add(x, y, w);
34                ins(x), ins(y);
35            }
36        }
37    } d1;
38    int *fa = d1.fa, &m = d1.m;
39    bool *typ = d1.typ;
```

2.5 二分栈

固定罚分下做“分段 DP 并提供前驱/段数”的斜率优化, $O(n \log n)$ 。

- `work(dt)`: 每段代价 `calc(j+1, i)` 减去罚分 `dt`, 求 `f[n]`(值 + 段数, `nd{cnt}`); 返回 `nd{f.x = 数值, f.c = 段数}`
- 数据结构: 单调栈 + 二分区段; `q` 维护尾点范围, `get(p, L)` 比较新旧决策

- 依赖: 全局 `N`(数组)与 `calc(l, r)` 函数
- 通常配 `WQS` 使用: 外层二分罚分, 这里给出对应的最优段数

坑: 要求 `calc` 满足四边形不等式(决策单调), 否则二分栈回退不正确; `nd` 的 `operator<` 是 (x, c) 字典序, 保证匹配时优先小罚分; `calc` 是 $O(1)$ 还是 $O(n)$ 需注意, 总复杂度将乘上它。

```
1 struct nd {
2     ll x;
3     int c;
4     bool operator<(const nd b) const {
5         return x < b.x || (x == b.x && c < b.c);
6     }
7     nd operator+(const nd b) const { return {x + b.x, c + b.c}; }
8 };
9 nd work(ll dt) {
10    static nd f[N];
11    deque<pii> q;
12    f[0] = {0, 0}, q.push_back({n, 0});
13    auto get = [&](int j, int i) -> nd {
14        return f[j] + nd{calc(j + 1, i) - dt, 1};
15    };
16    For(i, 1, n) {
17        f[i] = get(q.front()[1], i);
18        if(q.front()[0] <= i) q.pop_front();
19        while(q.size()) {
20            auto [R, p] = q.back(); q.pop_back();
21            int L = q.size() ? q.back()[0] + 1 : i + 1;
22            if(!(get(p, L) < get(i, L))) continue;
23            int l = L, r = R + 1;
24            while(l + 1 < r) {
25                int m = l + r >> 1;
26                get(p, m) < get(i, m) ? l = m : r = m;
27            }
28            q.push_back({l, p});
29            break;
30        }
31        q.push_back({n, i});
32    }
33    return f[n];
34 }
```

2.6 wqs 二分构造方案

外层 `WQS` 二分会惩罚值, 内层调用黑盒 `Wilber`(区间 DP 的裸段数相关结果), 最终返回分割点数组。

- 接口: `wqs(n, k, que, up)`; `que(l, r)` 为区间代价函数(返回 `i128`); `up` 为代价上界(二分范围 `-up..1`)
- 返回: 0 -based 分割点数组(以 0 起、 n 终), 恰含 k 个区间段
- 依赖: `Wilber` 类必须事先提供(该文件未给出)——通常是带决策单调性的最优区间划分解
- 复杂度: $O(\log C)$ 次内层 `Wilber`

坑: `WQS` 二分要求「段数更多的代价序列」满足凸性(否则失效); 内部 `que` 每段自动减去二分惩罚 `dt`, `que` 必须对任意 l, r 可控区间; `fp/gp` 数组拼结细节在 `get`, 若边界出错多为 `Wilber` 允许的段数与返回不符。

```

1 vect<int> wqs(int n, int k, auto &&que, i128 up) {
2     int B = __lg(n) + 1, T = (1 << B) - 1;
3     auto chk = [&](i128 dt) {
4         return Wilber<i128>(
5             n, [&](int l, int r) { return (que(l, r) - dt) << B | 1; }, 1e36);
6     };
7     auto get = [&](i128 dt) {
8         auto [f, fp] = chk(dt);
9         vect<int> pt;
10        for (int i = n; i; i = fp[i]) pt += i;
11        pt += 0;
12        reverse(all(pt));
13        return pt;
14    };
15    i128 l = -up - 1, r = 1;
16    while (l + 1 < r) {
17        i128 m = l + r >> 1;
18        (chk(m).first[n] & T) < k ? (l = m) : (r = m);
19    }
20    auto fp = get(l), gp = get(l + 1);
21    if (gp.size() - 1 == k) return gp;
22    int j = 1;
23    For(i, 1, gp.size() - 2) {
24        while (fp[j] < gp[i]) ++j;
25        if (fp[j - 1] <= gp[i - 1] && fp.size() - j + i - 1 == k) {
26            return gp.substr(0, i) + fp.substr(j);
27        }
28    }
29    assert(0);
30 }

```

2.7 O(1) RMQ

线性预处理后任意区间最值 $O(1)$ (常数很小, 可自定义比较器)。

- 模板参数: `rmq_t<T, cmp, dt>`; `cmp` 视作“取最优比较器”(less<T> 取 max, greater<T> 取 min); `dt` 是下标偏移 (默认 1 即 1-based 输入与查询)
- 接口: `bd(n, T* a) / bd(vector<T>&) / bd(n, func)(func(i) 从偏移处取值)`, 然后 `que(l, r)`
- 复杂度: 建 $O(n)$, 查 $O(1)$
- 依赖: 无 (自带模板参数)

坑: `st` 分块表仅对块间查询, `dt` 默认为 1 但 `bd(vector)` 传 `data() - dt` 指针, 注意边界; `cmp` 需要严格弱序; 长查询数据为 n 时块数 $(n-1)/B$ 别超 `st` 层数 (已按 `__lg` 计算)。

```

1 template <typename T, typename cmp = less<T>, int dt = 1>
2 struct rmq_t {
3     static constexpr int B = 20;
4     vector<T> a, pre, suf;
5     vector<vector<T>> st;
6     vector<int> f;
7     T _que(int L, int R) {
8         int k = __lg(R - L + 1);

```

```

9         return max(st[k][L], st[k][R - (1 << k) + 1], cmp());
10    }
11    T _que_bitc(int k, int L, int R) {
12        int l = k * B, x = f[R] & ~((1 << L - l) - 1);
13        return a[l + __lg(x & -x)];
14    }
15    void bd(int n, T *_a) {
16        a.resize(n), pre.resize(n), suf.resize(n), f.resize(n);
17        For(i, 0, n - 1) pre[i] = suf[i] = a[i] = _a[i + dt];
18        int k = (n - 1) / B;
19        st.resize(__lg(k + 1) + 1), st[0].resize(k + 1);
20        For(i, 0, k) {
21            int l = i * B, r = min(i * B + B - 1, n - 1);
22            rFor(i, r - 1, l) pre[i] = max(pre[i], pre[i + 1], cmp());
23            For(i, l + 1, r) suf[i] = max(suf[i], suf[i - 1], cmp());
24            st[0][i] = suf[r];
25            static int b[B + 1];
26            int t = 0, s = 0;
27            For(i, l, r) {
28                while(t && cmp()(a[b[t]], a[i])) s ^= 1 << b[t--] - l;
29                f[i] = s ^= 1 << i - l;
30                b[++t] = i;
31            }
32        }
33        For(i, 1, __lg(k + 1)) {
34            int m = k - (1 << i) + 1;
35            st[i].resize(m + 1);
36            For(j, 0, m) st[i][j] = max(st[i - 1][j], st[i - 1][j + (1 << i - 1)]),
37                cmp());
38        }
39        void bd(vector<T> &a) { bd(a.size(), a.data() - dt); }
40        void bd(int n, auto func) {
41            vector<T> a(n);
42            For(i, 0, n - 1) a[i] = func(i + dt);
43            bd(a);
44        }
45        T que(int l, int r) {
46            l -= dt, r -= dt;
47            int il = l / B, ir = r / B;
48            if(il == ir) {
49                return _que_bitc(il, l, r);
50            } else {
51                T s = max(pre[l], suf[r], cmp());
52                if(il + 1 < ir) s = max(s, _que(il + 1, ir - 1), cmp());
53                return s;
54            }
55        }
56    };

```

3 图论

3.1 网络流

Dinic 最大流,附赠残余网络割集。

- add(u, v, w) 加边; solve(s, t, n) 传点数并返回最大流; cut(f) 后 $f[i] = \frac{0}{1}$ 标记 $\frac{s}{t}$ 侧
- 边数上限 $N = 10^6$ (静态数组), 无穷大 $Z = 1e18$
- 复杂度 $O(n^2 m)$ 上界, 单位容量图 $O(m \sqrt{n})$

坑: bfs() 首行 $d[i] = -(i == s)$ 是笔误——只有 s 得 -1 , 其余 0 , 而 $!d[v]$ 只认 -1 当未访问, 导致 BFS 进不了任何点, 最大流恒为 0 (已实测)。改成 $d[i] = -1$ 即可。

```
1 class Dinic_t {
2     public:
3         static constexpr int N = 1e6 + 10;
4         static constexpr ll Z = 1e18;
5         struct edge {
6             int v, n;
7             ll w;
8         } e[N];
9         int n, hd[N], cur[N], d[N], tot = 1, s, t;
10        void _add(int u, int v, ll w) {
11            e[++tot] = {v, hd[u], w}, hd[u] = tot;
12        }
13        void add(int u, int v, ll w) {
14            _add(u, v, w), _add(v, u, 0);
15        }
16        void clear() {
17            For(i, 1, n) hd[i] = 0;
18            n = s = t = 0, tot = 1;
19        }
20        bool bfs() {
21            // d[s] = 0、其余 -1(未访问)。注意不能写成 d[i] = -(i == s):
22            // 那样邻居会拿到 0, 而下面的判空条件 !~d[v] 只认 -1, 于是 BFS 一个点都进不去。
23            For(i, 1, n) d[i] = -1, cur[i] = hd[i];
24            d[s] = 0;
25            queue<int> q;
26            q.push(s);
27            while (q.size()) {
28                int u = q.front();
29                q.pop();
30                if (u == t) return 1;
31                for (int i = cur[u]; i; i = e[i].n)
32                    if (e[i].w && !~d[e[i].v]) {
33                        d[e[i].v] = d[u] + 1, q.push(e[i].v);
34                    }
35            }
36            return 0;
37        }
38        ll dfs(int u, ll in) {
39            if (u == t) return in;
40            ll out = 0;
```

```
41            for (int &i = cur[u]; i; i = e[i].n)
42                if (e[i].w && d[e[i].v] == d[u] + 1) {
43                    ll s = dfs(e[i].v, min(in, e[i].w));
44                    e[i].w -= s, e[i ^ 1].w += s;
45                    in -= s, out += s;
46                    if (!in) break;
47                }
48            if (!out) d[u] = -1;
49            return out;
50        }
51        ll solve(int _s, int _t, int _n) {
52            s = _s, t = _t, n = _n;
53            ll ans = 0;
54            while (bfs()) ans += dfs(s, Z);
55            return ans;
56        }
57        void cut(int *f) { // 原来写的是 auto *f: C++20 语法, C++17 下只有 GCC 扩展(告警)
58            For(i, 1, n) f[i] = 1;
59            queue<int> q;
60            f[s] = 0;
61            q.push(s);
62            while (q.size()) {
63                int u = q.front();
64                q.pop();
65                for (int i = hd[u]; i; i = e[i].n)
66                    if (e[i].w && f[e[i].v]) {
67                        f[e[i].v] = 0;
68                        q.push(e[i].v);
69                    }
70            }
71        }
72    } f;
```

3.2 费用流

$O(f n \log m)$ 的势优化费用流, 以及能处理初始负环/下界的 mcmf2。

- add(u, v, w, c) 加容量 w 费用 c 的边; mcmf(s, t) 返回 {流量, 费用}; mcf = true 时费用为负(增广更优)才继续
- mcmf2(s, t): 先消除初始负费用边(翻转并按差额补平衡边), 适合有负环或下界流约束的图; 临时辅助边用完自动回收
- 依赖: 无; 容量流 f 由输入决定

坑: mcmf() 的初始势靠 SPFA, 原图有负环时报错/死循环, 只能用 mcmf2; mcf=1 表示“任意流中最小费用”而非固定流量。

```
1 struct MCMF_t {
2     static constexpr int N = 1e6 + 10, FL = 1e9;
3     static constexpr ll Z = 1e18;
4     struct edge {
5         int v, n, w;
6         ll c;
```

```

7   } e[N];
8   int n, mx, hd[N], tot = 1, s, t, p[N];
9   void _add(int u, int v, int w, ll c) {
10      e[++tot] = {v, hd[u], w, c}, hd[u] = tot;
11   }
12   void add(int u, int v, int w, ll c) {
13      _add(u, v, w, c), _add(v, u, 0, -c);
14   }
15   void clear() {
16      // 清到“用过的最大点数” mx:mcmf2() 的内层会用 _n + 2 个点跑,
17      // 只清到当前 n 的话,超级源/汇那条残留 hd[] 会让下一次调用拿陈旧前驱、
18      // dijkstra 回退时在 e[] 里乱跳 → 死循环(实测 3 节点图也挂)。
19      For(i, 1, mx) hd[i] = 0;
20      tot = 1, s = t = 0;
21   }
22   ll h[N], d[N];
23   void spfa() {
24      static bool vis[N];
25      For(i, 1, n) h[i] = Z;
26      queue<int> q;
27      q.push(s), h[s] = 0, vis[s] = 1;
28      while(q.size()) {
29         int u = q.front(), v;
30         q.pop();
31         vis[u] = 0;
32         for(int i = hd[u]; i; i = e[i].n)
33             if(e[i].w && h[u] + e[i].c < h[v = e[i].v]) {
34                 h[v] = h[u] + e[i].c;
35                 if(!vis[v]) q.push(v), vis[v] = 1;
36             }
37      }
38   }
39   bool dij() {
40      For(i, 1, n) d[i] = Z;
41      priority_queue<pair<ll, int>> q;
42      q.emplace(d[s] = 0, s);
43      int u;
44      while(q.size()) {
45         auto [tmp, u] = q.top();
46         q.pop();
47         if(-tmp != d[u]) continue;
48         for(int i = hd[u]; i; i = e[i].n)
49             if(e[i].w) {
50                 int v = e[i].v;
51                 auto w = d[u] + h[u] - h[v] + e[i].c;
52                 if(w < d[v]) d[v] = w, p[v] = i, q.emplace(-w, v);
53             }
54      }
55      For(i, 1, n) if(d[i] != Z) h[i] = d[i] += h[i];
56      return d[t] != Z;
57   }
58   pair<int, ll> mcmf(int _s, int _t, bool mcf = 0, int _n = 0) {
59      s = _s, t = _t, n = _n, cmax(mx, _n);

```

```

60      spfa();
61      int a1 = 0;
62      ll a2 = 0;
63      while(dij() && (!mcf || d[t] < 0)) {
64         int m = FL;
65         for(int u = t; u != s; u = e[p[u] ^ 1].v) m = min(m, e[p[u]].w);
66         a1 += m, a2 += m * d[t];
67         for(int u = t; u != s; u = e[p[u] ^ 1].v) e[p[u]].w -= m, e[p[u] ^ 1].w
+= m;
68      }
69      return {a1, a2};
70   }
71   pair<int, ll> mcmf2(int _s, int _t, bool mcf = 0, int _n = 0) {
72      s = _s, t = _t, n = _n, cmax(mx, _n + 2);
73      static int d[N];
74      int a1 = 0;
75      ll a2 = 0;
76      For(i, 2, tot) if(i % 2 == 0 && e[i].c < 0) {
77         int u = e[i ^ 1].v, v = e[i].v;
78         swap(e[i].w, e[i ^ 1].w);
79         d[u] -= e[i ^ 1].w, d[v] += e[i ^ 1].w;
80         a2 += e[i ^ 1].w * e[i].c;
81      }
82      For(i, 1, _n) {
83         d[i] > 0 ? add(_n + 1, i, d[i], 0) : add(i, _n + 2, -d[i], 0);
84      }
85      add(_t, _s, FL, 0);
86      int fake = tot, on = n; // 辅助边(_n+1 出入、_t -> _s)到此结束;记住原图的 n
87      // 这三处原来都有问题,是三个独立 bug:
88      // 1) 内层 mcmf 漏传 _n -> 内层 n = 0, spfa/dij 的 For(i, 1, n) 全空转 -> 死循环
89      // 2) e[tot].w 原来在 mcmf 之前读:那时 _t -> _s 上的流量还是 0,而且增广会往 e[]
里
90      // 追加反向边、下标 tot 之后还会变,所以可行性阶段的循环永远算不进来
91      // 3) tot -= 2 * (_n + 1) 与真实的辅助边条数对不上;另外内层跑完会把成员 n 改成
92      // _n + 2(并留下 hd[_n+1]、hd[_n+2]),不还原的话外层 mcmf 会带着残留走错路
93      // 辅助边条数 = 出边 _n + 入边 _n + _t -> _s 一条 + 超级源/汇两条 = 2 * _n + 2。
94      a2 += mcmf(_n + 1, _n + 2, 0, _n + 2).second;
95      a1 += e[fake].w;
96      tot = fake - 2 * _n - 2, n = on;
97      For(i, 1, on) while(hd[i] > tot) hd[i] = e[hd[i]].n;
98      auto [a3, a4] = mcmf(_s, _t, mcf, _n);
99      return {a1 + a3, a2 + a4};
100   }
101 };

```

3.3 支配树

$O((n + m) \alpha)$ 求有向图从根 1 出发的支配树。

- add(u, v) 加有向边; getdom(n) 一次求出, dm[u] 为 u 的直接支配点(根为 0)
- 内部 dfn 是 DFS 序; sd 半支配点
- 复杂度接近线性, 常数适中, $N = 10^6$

坑: 根固定为 1,图里所有点必须从 1 可达,否则未达点 dm 无意义;que() 是递归路径压缩,深图栈压力大时可改迭代。

```
1 namespace DOM {
2
3 const int N = 1e6 + 10;
4 int n;
5 vect<int> t[N], t2[N], q1[N];
6 int fa[N], f1[N], dfn[N], pos[N], dt, sd[N], dm[N];
7 pii f[N];
8 void add(int u, int v) {
9     t[u] += v, t2[v] += u;
10 }
11 void dfs(int u) {
12     pos[dfn[u] = ++dt] = u;
13     for(auto v : t[u])
14         if(!dfn[v]) {
15             dfs(v), f1[v] = u;
16         }
17 }
18 pii que(int u) {
19     if(fa[u] == u) return f[u];
20     cmin(f[u], que(fa[u]));
21     fa[u] = fa[fa[u]];
22     return f[u];
23 }
24 void getdom(int _n) {
25     n = _n;
26     // 只重置算法自身的状态,不动 t[]/t2[] (图是调用方建的,清了就没边可跑了)。
27     // 不重置的话同一进程内第二次调用会带着上一张图的 dfn/pos/dt 算错。
28     For(i, 1, n) q1[i].clear(), fa[i] = f1[i] = dfn[i] = pos[i] = sd[i] = dm[i] = 0;
29     dt = 0;
30     dfs(1);
31     For(i, 1, n) fa[i] = i, f[i] = {dfn[i], 0};
32     rFor(i, dt, 1) {
33         int u = pos[i], s = i;
34         for(auto x : q1[u]) dm[x] = que(x)[1];
35         if(i == 1) break;
36         for(auto x : t2[u])
37             if(dfn[x]) cmin(s, que(x)[0]); // 必须跳过从 1 不可达的前驱,否则 s 被拉成 0。
38         dm[u] 算错
39         sd[u] = pos[s];
40         q1[sd[u]] += u;
41         f[u] = {dfn[sd[u]], u};
42         for(auto v : t[u])
43             if(f1[v] == u) fa[v] = u;
44     }
45     For(i, 2, dt) {
46         int u = pos[i];
47         if(sd[dm[u]] == sd[u])
48             dm[u] = sd[u];
49         else
```

```
49         dm[u] = dm[dm[u]];
50     }
51 }
52
53 } // namespace DOM
```

3.4 一般图最大匹配

```
1 int n, fa[N];
2 vect<int> t[N];
3 int fd(int u) { return fa[u] == u ? u : fa[u] = fd(fa[u]); }
4 void getmat(int rt) {
5     static int vis[N], mat[N], pr[N];
6     For(i, 1, n) vis[i] = pr[i] = 0, fa[i] = i;
7     queue<int> q;
8     auto lca = [](int x, int y) {
9         static int vis[N], t; ++t;
10        while(1) {
11            if(x && exchange(vis[x = fd(x)], t) == t) return x;
12            x = pr[mat[x]], swap(x, y);
13        }
14        return -1;
15    };
16    auto blossom = [&](int x, int y, int z) {
17        for(; fd(x) != z; x = pr[y]) {
18            pr[x] = y, y = mat[x], fa[x] = fa[y] = z;
19            if(vis[y] == 1) vis[y] = 2, q.push(y);
20        }
21    };
22    vis[rt] = 2, q.push(rt);
23    while(q.size()) {
24        int u = q.front(); q.pop();
25        for(auto v : t[u]) {
26            // 关键:同一朵花(同一并查集分量)内的边要跳过。少了这一行,花缩起来之后
27            // 花内相邻的点还会互相触发下面的分支,导致 mat[] 被写脏并死循环
28            // (实测:400 组随机 n=6..12 的图里有 4 组卡死;补上后 0 挂死 0 错)。
29            if(fd(u) == fd(v)) continue;
30            if(!vis[v]) pr[v] = u, vis[v] = 1, vis[mat[v]] = 2, q.push(mat[v]);
31            if(!mat[v]) {
32                while(v) mat[v] = pr[v], swap(mat[pr[v]], v);
33                return;
34            }
35            if(vis[v] == 2) {
36                int l = lca(u, v);
37                blossom(u, v, l), blossom(v, u, l);
38            }
39        }
40    }
41 }
```

4 数学

4.1 Barret 约简

```
1 struct barret {
2     u128 w; u64 M;
3     barret(const u64 M) : M(M), w(u64((u128(1) << 64) / M)) {}
4     inline u64 mod(u64 x) const {
5         x -= ((w * x) >> 64) * M;
6         return x >= M ? x - M : x;
7     }
8     inline u64 mod_without_chk(u64 x) const { return x - ((w * x) >> 64) * M; }
9 };
```

4.2 拉格朗日插值

已知 $n+1$ 个点 $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$, 其中 x_i 互不相同, 则存在唯一的次数不超过 n 的多项式 $L(x)$ 满足 $L(x_i) = y_i$.

Lagrange 插值多项式为:

$$L(x) = \sum_{i=0}^n y_i \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j}$$

当 x_i 是连续整数 (如 $0, 1, \dots, n$) 时, 可 $O(n)$ 计算 $L(x)$ (需预处理阶乘逆元)。此时:

$$L(x) = \sum_{i=0}^n y_i * \frac{\prod_{j=0, j \neq i}^n (x - j)}{(-1)^{n-i} i! (n-i)!}$$

其中分子可通过前后缀积 $O(1)$ 得到。

```
1 vector<mint> lag(vector<pair<mint, mint>> a) {
2     int n = a.size();
3     vector<mint> f(n + 1), ans(n + 1, 0);
4     f[0] = 1;
5     For(i, 1, n) {
6         auto [x, y] = a[i - 1];
7         rFor(j, i - 1, 0) f[j + 1] += f[j], f[j] *= -x;
8     }
9     For(i, 1, n) {
10        auto [x, y] = a[i - 1];
11        mint w = 1, inv = !x ? 0 : -x.inv(), z = 0;
12        For(j, 1, n) if(i != j) w *= x - a[j - 1].first;
13        w = w.inv() * y;
14        if(!x) For(j, 1, n) ans[j - 1] += f[j] * w;
15        else For(j, 0, n) z = (f[j] - z) * inv, ans[j] += z * w;
16    }
17    return ans;
18 }
19
20 // 需要 fac(n), ifac(n)
21 void lag_i(int n, mint *a) {
22     vector<mint> f(n + 1, 0), g(n, 0);
```

```
23     f[0] = 1;
24     For(i, 1, n) rFor(j, i - 1, 0) f[j + 1] += f[j], f[j] *= mint::raw(mint::getM()
25     - i);
26     For(i, 1, n) {
27         mint inv = -ifac(i) * fac(i - 1), w = a[i] * ifac(n - i) * ifac(i - 1) * ((i
28         ^ n) & 1 ? -1 : 1), s = 0;
29         For(j, 0, n - 1) s = (f[j] - s) * inv, g[j] += s * w;
30     }
31 }
32 mint lag_p(int n, mint *a, mint x) {
33     if(x.val() < n) return a[x.val()];
34     vector<mint> f(n + 1), g(n + 1);
35     mint ans = 0;
36     f[0] = g[n] = 1;
37     For(i, 0, n - 1) f[i + 1] = f[i] * (x - i);
38     rFor(i, n - 1, 0) g[i] = g[i + 1] * (x - i);
39     For(i, 0, n - 1) {
40         mint w = f[i] * g[i + 1] * a[i] * ifac(n - 1 - i) * ifac(i) * ((i ^ n) & 1 ?
41         1 : -1);
42         ans += w;
43     }
44     return ans;
```

4.3 Pollard Rho

```
1 constexpr bool chkp(ll n) {
2     constexpr int p[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022}; // {2,
3     auto ksm = [&](i128 a, ll b) {
4         i128 s = 1;
5         for(; b >>= 1, a = a * a % n) b & 1 && (s = s * a % n, 0);
6         return s;
7     };
8     if(n <= 40)
9         return binary_search(p, p + sizeof(p), n);
10    else {
11        if(n % 2 == 0) return 0;
12        int t = __builtin_ctzll(n - 1), i = 0;
13        for(auto b : p) {
14            i128 x = ksm(b, n - 1 >> t);
15            if(x == 1) continue;
16            for(i = 0; i < t && x != n - 1; ++i) x = x * x % n;
17            if(i == t) return 0;
18        }
19        return 1;
20    }
21 }
22 ll PR(ll n) {
23     mt19937_64 rng(114514);
24     if(n % 2 == 0) return 2;
```

```

25 static constexpr int S = 127;
26 uniform_int_distribution<> rnd(1, n - 1);
27 ll x = 0, y = 0, c = rnd(rng), w = 1, g;
28 auto f = [&](ll x) { return (i128(x) * x + c) % n; };
29 for(int t = 1;; ++t) {
30     x = f(x), y = f(f(y));
31     if(x == y) {
32         x = y = 0, c = rnd(rng);
33         continue;
34     }
35     w = i128(w) * abs(x - y) % n;
36     if(!w) return __gcd(abs(x - y), n);
37     if(!(y & S) && (g = __gcd(w, n)) > 1) return g;
38 }
39 }
40 vect<ll> fact(ll n) {
41     if(n == 1) return {};
42     vect<ll> f;
43     auto S = [&](ll n, auto &&S) {
44         if(chkp(n)) return f += n, void();
45         ll x = PR(n);
46         while(n % x == 0) n /= x;
47         S(x, S);
48         if(n > 1) S(n, S);
49     };
50     S(n, S);
51     sort(all(f)), f.erase(unique(all(f)), f.end());
52     return f;
53 }

```

4.4 min_25

$O(N^{0.75} / \log N)$ 求积性函数 f 的前缀和,要求 f 在素数点 $f(p)$ 是关于 p 的低次多项式(最后拆成 p 的幂的线性组合)。

- 接口: solve(n),内部依次 init()(线性筛出 10^5 内质数与 $s1/s2/s: \Sigma p, \Sigma p^2, \Sigma(p^2-p)$)+ solveg(n) (求 g 数组)+ $f(n, m)$ 递归求和
- func(x): 素数幂 $x = p^k$ 处 f 的取值,题变则改(示例 $x(x-1)$ 即 $f(p) = p(p-1)$,欧拉函数类)
- 上限: $\text{sqrt}(n) \leq 10^5$,即 $n \leq 10^{10}$;模数固定 $M = 1e9 + 7$
- g 数组大小 $2e5$ (双值映射 $\text{id}(n)$)

坑: solve 内部已调 init(),重复使用只需再次 solve;sqrt(n) 为整平方时内部 fl 有特殊处理,别乱改;数组 g1/g2 是模 M 下的量。

```

1 const int M = 1e9 + 7;
2 constexpr int i2 = M + 1 >> 2, i6 = (M + 1) / 6;
3 void add(int &a, int b) {
4     a += b;
5     a >= M && (a -= M);
6 }
7 void sub(int &a, int b) {
8     a -= b;

```

```

9     a < 0 && (a += M);
10 }
11 const int N = 1e5 + 10;
12 int p[N], pc, p2[N];
13 int s1[N], s2[N], s[N];
14 void init() {
15     static const int n = 1e5;
16     static int b[N];
17     For(i, 2, n) {
18         if(!b[i]) p[++pc] = i;
19         For(j, 1, pc) if(i * p[j] <= n) {
20             b[i * p[j]] = 1;
21             if(i % p[j] == 0) break;
22         }
23         else break;
24     }
25     p[pc + 1] = n + 1;
26     For(i, 1, pc) {
27         p2[i] = 1ll * p[i] * p[i] % M;
28         s1[i] = (s1[i - 1] + p[i]) % M;
29         s2[i] = (s2[i - 1] + p2[i]) % M;
30         s[i] = (s2[i] - s1[i] + M) % M;
31     }
32 }
33 ll lim, n0;
34 int g1[N * 2], g2[N * 2];
35 int id(ll n) {
36     return n <= lim ? n : n0 / n + lim;
37 }
38 void solveg(ll _n0) {
39     n0 = _n0;
40     lim = sqrt(n0);
41     ll fl = n0 / lim == lim;
42     auto init = [&](ll n, int _id) {
43         n %= M;
44         sub(g1[_id] = n * (n + 1) / 2 % M, 1);
45         sub(g2[_id] = n * (n + 1) % M * (n + n + 1) % M * i6 % M, 1);
46     };
47     auto doit = [&](ll n, int m, int i1, int i2) {
48         sub(g1[i1], 1ll * (g1[i2] - s1[m - 1] + M) * p[m] % M);
49         sub(g2[i1], 1ll * (g2[i2] - s2[m - 1] + M) * p2[m] % M);
50     };
51     For(i, 1, lim) init(i, i);
52     For(i, 1, lim - fl) init(n0 / i, i + lim);
53     For(i, 1, pc) {
54         ll x = p[i], y = x * x, z = n0 / x;
55         For(j, 1, min(lim - fl, n0 / y)) doit(n0 / j, i, j + lim, id(z / j));
56         rFor(j, lim, min(lim + 1, y)) doit(j, i, j, j / x);
57     }
58 }
59 int g(ll n) { return (g2[id(n)] - g1[id(n)] + M) % M; }
60 ll func(ll n) {
61     n %= M;

```

```

62     return n * (n - 1) % M;
63 }
64 int f(ll n, int m) {
65     if(p[m] > n) return 0;
66     ll ans = g(n) - s[m] + M;
67     For(j, m + 1, pc) {
68         int p = ::p[j];
69         if(1ll * p * p > n) break;
70         for(ll s = p; s * p <= n; s *= p) {
71             ans += func(s) * f(n / s, j) % M + func(s * p);
72         }
73     }
74     return ans % M;
75 }
76 ll solve(ll n) {
77     init(), solveg(n);
78     return (f(n, 0) + 1) % M;
79 }

```

4.5 类欧

$O(\log n)$ 求 $\sum_{i=0}^n \lfloor (a \cdot i + b) / c \rfloor$ 及其平方、带权。

- 结构版 nd f(int n, int a, int b, int c): 参数顺序 (n, a, b, c) , 一次递归同时得到 $\{f, g, h\}$:
 $\triangleright f = \sum \lfloor (ai+b)/c \rfloor; g = \sum \lfloor \cdot \rfloor^2; h = \sum i \cdot \lfloor (ai+b)/c \rfloor$
- 单值版 f(int a, int b, int c, int n): 参数顺序 (a, b, c, n) , 只求 f
- 依赖: mint(带 inv 等)、全局 i2/i6

坑: 两个 f 重载签名都是 (int, int, int, int), C++ 按返回类型不计入签名, 不能同时编译——实际用哪版复制哪版, 并留心两版参数顺序相反; 递归深度 $O(\log n)$, 无栈风险。

```

1 const mint i2 = mint(2).inv(), i6 = mint(6).inv();
2 struct nd {
3     mint f, g, h;
4 };
5 nd f(int n_, int a, int b, int c) {
6     mint n = n_, x = a / c, y = b / c, s0 = n + 1, s1 = n * (n + 1) * i2, s2 = n *
    (n + 1) * (n + n + 1) * i6;
7     if(!a) return {s0 * y, s0 * y * y, s1 * y};
8     if(a >= c || b >= c) {
9         nd rs = f(n_, a % c, b % c, c);
10        mint f = rs.f, g = rs.g, h = rs.h;
11        return {f + s1 * x + s0 * y, g + s2 * x * x + s0 * y * y + (x * h + y * f +
    s1 * x * y) * 2, h + s2 * x + s1 * y};
12    }
13    int m = (1ll * a * n_ + b) / c;
14    nd rs = f(m - 1, c, c - b - 1, a);
15    mint f = rs.f, g = rs.g, h = rs.h;
16    return {n * m - f, n * m * m - h * 2 - f, i2 * (n * n * m + n * m - g - f)};
17 }
18 mint f(int a, int b, int c, int n) {
19     if(!a) return mint(b / c) * (n + 1);
20     if(a >= c || b >= c) {

```

```

21     mint x;
22     if(n & 1)
23         x = mint(n + 1 >> 1) * n * (a / c);
24     else
25         x = mint(n >> 1) * (n + 1) * (a / c);
26     return f(a % c, b % c, c, n) + x + mint(n + 1) * (b / c);
27 }
28 int m = (1ll * a * n + b) / c;
29 return mint(n) * m - f(c, c - b - 1, a, m - 1);
30 }

```

5 数论

5.1 分数还原

$q \equiv \frac{x}{a} \pmod{p}$, $x \leq A$, $|a|$ 取到最小值

```

1 pii approx(int p, int q, int A) {
2     int x = q, y = p, a = 1, b = 0;
3     while(x > A) {
4         swap(x, y), swap(a, b), a -= x / y * b, x %= y;
5     }
6     return {x, a};
7 }

```

5.2 $O(V)$ - $O(1)$ GCD

```

1 struct fgcd_t {
2     static constexpr int n = 1e7, B = 3163;
3     array<int, 3> a[n + 1];
4     int s[B + 1][B + 1];
5     bitset<n + 1> b;
6     fgcd_t() {
7         vect<int> p;
8         a[1] = {1, 1, 1};
9         For(i, 2, n) {
10             if(!b[i]) i <= B && (p += i, 0), a[i] = {i, 1, 1};
11             for(auto z : p) {
12                 int x = i * z; if(x > n) break;
13                 b[x] = 1, a[x] = a[i];
14                 *min_element(all(a[x])) *= z;
15                 if(i % z == 0) break;
16             }
17         }
18         For(i, 0, B) For(j, 0, i) s[i][j] = !j ? i : s[j][i % j];
19     }
20     int operator()(int x, int y) const {
21         if(!x || !y) return x | y;
22         if(x == 1 || y == 1) return 1;
23         auto g1 = [&](int x, int y) {
24             int t;
25             if(y == 1 || (t = x % y) == 0) return y;

```

```

26         return y > B ? 1 : s[y][t];
27     };
28     int a0 = g1(x, a[y][0]), a1 = g1(x /= a0, a[y][1]), a2 = g1(x /= a1, a[y]
[2]);
29     return a0 * a1 * a2;
30 }
31 };

```

5.3 离散对数

```

1 ll BSGS(ll a, ll b, ll p) {
2     a %= p, b %= p;
3     if(p == 1) return 0;
4     if(b == 1) return 0; // a^0 = 1
5     if(!a) return -1;
6     // l = a 的阶: l | p - 1, 从 p - 1 出发按素因子反复试除
7     ll l = p - 1;
8     for(ll q = 2; q * q <= l; ++q)
9         for(; l % q == 0 && ksm(a, l / q, p) == 1; l /= q);
10    for(; l > 1 && ksm(a, l - 1, p) == 1; l = 1; // 剩下的素因子最多一个
11    // m = ceil(sqrt(l)), 保证 m * m >= l; 大解落在 (m, m * m) 里
12    ll m = (ll) sqrt((db) l);
13    while (m * m < l) ++m;
14    for(ll y = 0, w = 1 % p; y < std::min(m, l); ++y) { // y < m 的小解
15        if(w == b) return y;
16        w = (i128) w * a % p;
17    }
18    // 大解写成 y = i * m + j (i >= 1, 0 <= j < m): a^j = b * a^(-i * m), 查表中即得
19    unordered_map<ll, ll> pos;
20    pos.reserve(m + 2);
21    for(ll j = 0, w = 1 % p; j < m; ++j) {
22        pos.emplace(w, j);
23        w = (i128) w * a % p;
24    }
25    ll step = ksm(a, m, p), iv = ksm(step, p - 2, p), w = (i128) b * iv % p, ans =
-1;
26    for(ll i = 1; i <= m; ++i, w = (i128) w * iv % p)
27        if(auto it = pos.find(w); it != pos.end()) {
28            ll y = (ll) i * m + it->second;
29            if(ksm(a, y, p) == b && (ans < 0 || y < ans)) ans = y; // 显式验证, 稳一手
30        }
31    return ans < 0 ? -1 : ans;
32 }

```

5.4 中国剩余定理

```

1 ll CRT(ll a1, ll p1, ll a2, ll p2) {
2     ll x, y, g = exgcd(p1, p2, x, y);
3     if((a2 - a1) % g) return -1;
4     ll m1 = p1 / g, m2 = p2 / g;
5     // p1 * t ≡ a2 - a1 (mod p2) 约去 g 后: p1 / g * t ≡ (a2 - a1) / g (mod m2)
6     ll k = (a2 - a1) / g % m2 * (x % m2 + m2) % m2;

```

```

7     ll ans = ((i128) k * p1 + a1) % (m1 * p2); // mod lcm 得到 [0, lcm) 内的解
8     return (ans + m1 * p2) % (m1 * p2);
9 }
10 // 多方程: ans = 0, mod = 1;
11 // For(i, 1, n) {
12 //     ans = CRT(ans, mod, a[i], p[i]);
13 //     if(ans < 0) 无解;
14 //     mod = mod / gcd(mod, p[i]) * p[i];
15 // }

```

5.5 Miller-Rabin 素性检验

```

1 namespace MR {
2     u64 mul(u64 a, u64 b, u64 p) { return (unsigned __int128) a * b % p; }
3     u64 pw(u64 a, u64 x, u64 p, u64 r = 1) {
4         for(; x >= 1, a = mul(a, a, p))
5             if(x & 1) r = mul(r, a, p);
6         return r;
7     }
8 }
9 // 底数取前 12 个素数: 对 u64 范围是确定性判定, 没有错误率。
10 bool chkp(u64 p) {
11     if(p < 2) return 0;
12     if(p < 4) return 1;
13     for(u64 d : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
14         if(p % d == 0) return p == d;
15     u64 s = __builtin_ctzll(p - 1), d = p - 1 >> s;
16     for(u64 a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
17         u64 x = MR::pw(a, d, p);
18         if(x == 1 || x == p - 1) continue;
19         bool ok = 0;
20         For(i, 1, s - 1) {
21             x = MR::mul(x, x, p);
22             if(x == p - 1) { ok = 1; break; }
23         }
24         if(!ok) return 0;
25     }
26     return 1;
27 }

```

5.6 Pollard-Rho 分解质因数

```

1 u64 isqrt(u64 x) {
2     if(x < 2) return x;
3     u64 r = sqrtl((long double) x);
4     for(; (unsigned __int128) r * r > x; --r);
5     for(; (unsigned __int128) (r + 1) * (r + 1) <= x; ++r);
6     return r;
7 }
8 // 返回 x 的一个非平凡因子; 依赖 chkp (即「Miller-Rabin」模板)。
9 // 先摘掉小因子与完全平方数: 这两种情形会让 rho 只给出平凡因子、空转。
10 u64 rho(u64 x) {

```

```

11     if(x % 2 == 0) return 2;
12     if(x % 3 == 0) return 3;
13     For(d, 5, 100) if(x % d == 0) return d;
14     if(u64 s = isqrt(x); s * s == x) return s;
15     for(; !chkp(x);) {
16         u64 c = 1 + rand() % x, t = 0, r = 0, p = 1;
17         auto f = [&](u64 v) { return ((unsigned __int128) v * v + c) % x; };
18         for(; p == 1; p = gcd(t > r ? t - r : r - t, x)) t = f(t), r = f(r);
19         if(p != x) return p; // 平凡因子:换个 c 重来
20     }
21     return x;
22 }
23 // 把 x 的质因子按从小到大存入 res(带重数)。期望复杂度  $O(x^{1/4})$ 。
24 void fact(u64 x, vect<u64> &res) {
25     if(x == 1) return;
26     if(chkp(x)) return res.push_back(x), void();
27     u64 d = rho(x);
28     fact(d, res), fact(x / d, res);
29 }

```

5.7 扩展欧几里得

```

1 ll exgcd(ll a, ll b, ll &x, ll &y) {
2     if(!b) return x = 1, y = 0, a;
3     ll g = exgcd(b, a % b, y, x);
4     return y -= a / b * x, g;
5 }
6 // 求 a 在模 p 下的逆元(要求 gcd(a, p) = 1):
7 // ll x, y; exgcd(a, p, x, y); x = (x % p + p) % p;

```

5.8 最小模线性值

```

1 int minmod(int n, int m, int a, ll b) {
2     int ans = m;
3     for(; n; swap(a, m)) {
4         a %= m, b %= m;
5         if(b < 0) b += m;
6         cmin(ans, (int) b);
7         n = ((ll) (n - 1) * a + b) / m;
8         b -= (ll) m * n;
9     }
10    return ans;
11 }

```

5.9 二次剩余

```

1 namespace QR {
2     ll mul(ll a, ll b, ll p) { return (ll) ((unsigned __int128) a * b % p); }
3     ll pw(ll a, ll x, ll p, ll r = 1) {
4         for(a %= p, (a += p) %= p; x; x >>= 1, a = mul(a, a, p))
5             if(x & 1) r = mul(r, a, p);
6         return r;
7     }
8 }

```

```

7     }
8 }
9 // quadres(a, p): 判断 a 是否为奇素数 p 的二次剩余(要求  $0 \leq a < p$ )。
10 bool quadres(u64 a, u64 p) {
11     return !a || QR::pw(a, (p - 1) >> 1, p) == 1;
12 }
13 // sqrt(x, p): 求 x 在模奇素数 p 下的平方根(要求  $0 \leq x < p$ )。
14 // p 模 4 余 3 时直接算,否则用 Cipolla;x 非二次剩余时返回 -1。
15 ll sqrt(ll x, ll p) {
16     if(x <= 1) return x;
17     if(QR::pw(x, (p - 1) >> 1, p) != 1) return -1;
18     if(p % 4 == 3) return QR::pw(x, (p + 1) >> 2, p);
19     static mt19937_64 gen;
20     ll w, a;
21     do a = gen() % p; while (!a || QR::pw(w = (QR::mul(a, a, p) - x % p + p) % p, (p - 1) >> 1, p) != p - 1);
22     auto prod = [&](pll u, pll v) { //  $F_p[\sqrt{w}]$  里的乘法:  $(u_0 + u_1\sqrt{w})(v_0 + v_1\sqrt{w})$ 
23         ll t0 = (QR::mul(u.first, v.first, p) + QR::mul(QR::mul(u.second, v.second, p), w, p)) % p;
24         ll t1 = (QR::mul(u.first, v.second, p) + QR::mul(u.second, v.first, p)) % p;
25         return make_pair(t0, t1);
26     };
27     pll r{1, 0}, b{a, 1};
28     for(ll k = (p + 1) >> 1; k; k >>= 1, b = prod(b, b))
29         if(k & 1) r = prod(r, b);
30     return r.first;
31 }

```

5.10 原根与阶

```

1 int root(int p) {
2     if(p == 2) return 1;
3     static vect<int> fac;
4     fac = {}, fac.reserve(20);
5     int x = p - 1;
6     for(int i = 2; i * i <= x; ++i)
7         if(x % i == 0) {
8             fac.push_back(i);
9             while (x % i == 0) x /= i;
10        }
11    if(x > 1) fac.push_back(x);
12    For(g, 2, p - 1) {
13        bool ok = 1;
14        for(auto q : fac) ok &= ksm(g, (p - 1) / q, p) != 1;
15        if(ok) return g;
16    }
17    return -1;
18 }
19 // ord(a, p): a 模素数 p 的阶,即最小的  $d > 0$  使  $a^d \equiv 1 \pmod{p}$ 。要求  $\gcd(a, p) = 1$ 。
20 // 从  $p - 1$  出发,对  $p - 1$  的每个素因子 q 反复试除(能除动说明阶里不含 q),
21 // 除到除不动为止,剩下的就是阶。
22 int ord(int a, int p) {

```

```

23     if(p == 2) return 1;
24     int x = p - 1;
25     static vect<int> fac;
26     int y = x;
27     fac = {}, fac.reserve(20);
28     for(int i = 2; i * i <= y; ++i)
29         if(y % i == 0) {
30             fac.push_back(i);
31             while (y % i == 0) y /= i;
32         }
33     if(y > 1) fac.push_back(y);
34     for(auto q : fac)
35         while (x % q == 0 && ksm(a, x / q, p) == 1) x /= q;
36     return x;
37 }

```

5.11 杜教筛

```

1 unordered_map<ll, ll> smu_big, sphi_big;
2 ll DJS_mu(ll n) { // f = mu, 取 g = 1, 则 H(n) = 1
3     if(n <= N) return smu[n];
4     if(smu_big.count(n)) return smu_big[n];
5     ll ans = 1;
6     for(ll l = 2, r; l <= n; l = r + 1)
7         r = n / (n / l), ans -= (r - l + 1) * DJS_mu(n / l);
8     return smu_big[n] = ans;
9 }
10 ll DJS_phi(ll n) { // f = phi, 取 g = 1, 则 H(n) = n * (n + 1) / 2
11     if(n <= N) return sphi[n];
12     if(sphi_big.count(n)) return sphi_big[n];
13     ll ans = (ll) n * (n + 1) / 2; // 先把 n 转成 ll, 否则 n 到 1e6 就溢出
14     for(ll l = 2, r; l <= n; l = r + 1)
15         r = n / (n / l), ans -= (r - l + 1) * DJS_phi(n / l);
16     return sphi_big[n] = ans;
17 }
18 // 通用形式: 只需实现 f 与 g, 其余照抄上面两个函数的结构
19 // ll S(ll n) {
20 //     if(n <= N) return pre[n];
21 //     if(mp.count(n)) return mp[n];
22 //     ll ans = H(n); // f * g 的前缀和
23 //     for(ll l = 2, r; l <= n; l = r + 1)
24 //         r = n / (n / l), ans -= (g_pre(r) - g_pre(l - 1)) * S(n / l);
25 //     return mp[n] = ans;
26 // }

```

5.12 线性筛

```

1 struct sieve_t {
2     int p[N / 10 + 5], tot, mn[N + 1], phi[N + 1], mu[N + 1], d[N + 1], cnt[N + 1],
3     lpf[N + 1];
4     ll s[N + 1];
5     void work() {

```

```

5         tot = 0, phi[1] = mu[1] = d[1] = s[1] = 1;
6         For(i, 2, N) {
7             if(!mn[i])
8                 p[++tot] = i, mn[i] = i, phi[i] = i - 1, mu[i] = -1,
9                 d[i] = 2, cnt[i] = 1, s[i] = i + 1, lpf[i] = i + 1;
10            for(int j = 1, x; j <= tot && (x = i * p[j]) <= N; ++j) {
11                mn[x] = p[j], cnt[x] = 1, lpf[x] = p[j] + 1;
12                if(i % p[j] == 0) {
13                    phi[x] = phi[i] * p[j], mu[x] = 0;
14                    cnt[x] = cnt[i] + 1, lpf[x] = lpf[i] * p[j] + 1;
15                    d[x] = d[i] / (cnt[i] + 1) * (cnt[x] + 1);
16                    s[x] = s[i] / lpf[i] * lpf[x];
17                    break;
18                }
19                phi[x] = phi[i] * (p[j] - 1), mu[x] = -mu[i];
20                d[x] = d[i] * 2, s[x] = s[i] * (p[j] + 1);
21            }
22        }
23    }
24    bool isp(int x) const { return x >= 2 && mn[x] == x; }
25 } S;
26 // 莫比乌斯/欧拉函数的前缀和(杜教筛要用): 筛完后
27 // For(i, 1, N) smu[i] = smu[i - 1] + S.mu[i], sphi[i] = sphi[i - 1] + S.phi[i];

```

5.13 下取整和

求 $\sum_{i=0}^{n-1} \lfloor (a \cdot i + b) / m \rfloor$, 复杂度 $O(\log m)$ 。参数全是无符号, a 、 b 不必小于 m 。

用法

- 直接调用: `floor_sum(n, m, a, b)`, 返回上面那个和; $n = 0$ 时返回 0。
- 求的是从 $i = 0$ 起的 n 项。若题目是 $\sum_{i=1}^n$, 写成 `floor_sum(n + 1, m, a, b)` 或先扣掉 $i = 0$ 的 b / m 。
- 从 l 到 r 的区间和: `floor_sum(r + 1, m, a, b) - floor_sum(l, m, a, b)`。
- 需要 $\sum_{i=0}^{n-1} \lfloor (a \cdot i + b) \rfloor$ 这类“上取整/整除”时, 先化成下取整再套本模板。

契约与坑

- 答案本身不能超过 $2^{64} - 1$ (累加器是 u64, 只有中间量 y 用了 i128): 大致 $n^2 \cdot a / m$ 量级, $n \leq 1e9$ 、 $a < m \leq 1e9$ 时安全; 更大的话把返回类型与 `ans` 一并换成 `unsigned __int128`。
- 参数是无符号的, 别传负数(负数下取整得自己先按 m 归正)。
- 它不等于数学章那个「类欧」: 类欧.cpp 是求 $\lfloor (ai+b)/c \rfloor$ 的 **f/g/h 三个量** 的递归模板, 本模板只求那一项下取整和。

```

1 u64 floor_sum(u64 n, u64 m, u64 a, u64 b) {
2     u64 ans = 0;
3     for(;;) {
4         if(a >= m) ans += n * (n - 1) / 2 * (a / m), a %= m;
5         if(b >= m) ans += n * (b / m), b %= m;
6         u64 y = (i128) a * n + b;

```

```

7     if(y < m) break;
8     n = y / m, b = y % m, swap(m, a);
9 }
10 return ans;
11 }

```

6 多项式

6.1 任意模数 NTT

```

1 namespace Mul {
2 using poly = vector<int>;
3 template <int M, int G> static void NTT(poly &a, const int k) {
4     auto ksm = [&](ll a, int b) {ll
ans=1;for(;b;b>>=1,a=a*a%M)if(b&1)ans=ans*a%M;return ans; };
5     static poly r, w;
6     int n = a.size();
7     if(n != r.size()) {
8         int l = __lg(n);
9         r.resize(n);
10        For(i, 0, n - 1) r[i] = (r[i >> 1] >> 1) | ((i & 1) << (l - 1));
11    }
12    For(i, 0, n - 1) if(i < r[i]) swap(a[r[i]], a[i]);
13    ll wi = ksm(G, (M - 1 >> 1) / n * k + M - 1);
14    w.resize(n), w[0] = 1;
15    For(i, 1, n - 1) w[i] = w[i - 1] * wi % M;
16    for(int m = 1, l = __lg(n); m < n; m <= 1, --l)
17        for(int i = 0; i < n; i += m + m)
18            For(j, 0, m - 1) {
19                ll x = a[i + j], y = 1ll * w[j << l] * a[i + j + m] % M;
20                a[i + j] = x + y >= M ? x + y - M : x + y;
21                a[i + j + m] = x - y < 0 ? x - y + M : x - y;
22            }
23    if(k == -1) {
24        ll inv = ksm(n, M - 2);
25        For(i, 0, n - 1) a[i] = a[i] * inv % M;
26    }
27 }
28 poly mul(poly a, poly b, int P) {
29     int n = a.size(), m = b.size();
30     if(!n || !m) return poly();
31     int l = 1 << __lg(n + m - 2) + 1;
32     a.resize(l), b.resize(l);
33     static constexpr int M[3]{998244353, 1004535809, 469762049}, G = 3;
34 #define g(p) \
35     [&](poly a, poly b) { \
36         NTT<M[p], G>(a, 1), NTT<M[p], G>(b, 1); \
37         For(i, 0, l - 1) a[i] = 1ll * a[i] * b[i] % M[p]; \
38         NTT<M[p], G>(a, -1); \
39         return a; \
40     }(a, b)
41     poly c[3]{g(0), g(1), g(2)}, d;

```

```

42     d.reserve(n + m - 1);
43 #undef g
44     static constexpr ll m01 = 1ll * M[0] * M[1];
45     static constexpr int i0 = 669690699, i1 = 354521948;
46     For(id, 0, n + m - 2) {
47         auto A = c[0][id], B = c[1][id], C = c[2][id];
48         ll x = 1ll * (B - A + M[1]) % M[1] * i0 % M[1] * M[0] + A;
49         d.push_back((1ll * (C - x % M[2] + M[2]) % M[2] * i1 % M[2] * (m01 % P) % P
+ x) % P);
50     }
51     return d;
52 }
53 } // namespace Mul
54 using poly = vector<mint>;
55 int M;
56 poly mul(poly a, const poly &b) {
57     int n = a.size(), m = b.size();
58     if(!n || !m) return poly();
59     vector<int> c(n), d(m);
60     For(i, 0, n - 1) c[i] = a[i].x;
61     For(i, 0, m - 1) d[i] = b[i].x;
62     c = Mul::mul(c, d, M);
63     a.resize(c.size());
64     For(i, 0, a.size() - 1) a[i] = c[i];
65     return a;
66 }

```

6.2 多项式复合

模 998244353, $F(G(x)) \bmod x^n$, 一次调用全部解出。

- 完整程序(自带 main): 第一行 n , 随后两行系数; 输出复合结果 n 个系数
- comp(F, G, n): 接口, 要求 $[x^0]G = 0$; prep(22) 一次性预处理 NTT 单位根(支持长度 $\leq 2^{22}$)
- 内部 solve(F, G, n, m): 分治卷积, 先乘 $A \cdot B$ 再做奇偶抽取
- 复杂度: 多项式级(小常数大), 适合 $n \leq 10^5$ 量级

坑: 数组全局且巨大($4.4e6 * \text{十几个}$), 内存约 300+ MB, 跑题前确认空间; scanf/printf 输入输出; F 与 G 都从常数项开始读入。

```

1 typedef vector<int> vi;
2 typedef vector<vi> vv;
3 typedef long long ll;
4 typedef unsigned long long ull;
5 const int N = 4.4e6 + 15, mod = 998244353;
6 int F[N], X[N], Y[N], W[N];
7 ull Z[N];
8 int rev[N * 2], omg[N * 2];
9 int qpow(int x, int y)
10 {
11     int ans = 1;
12     while(y)
13     {
14         if(y & 1)

```



```

15     ans = (ll)ans * x % mod;
16     x = (ll)x * x % mod;
17     y >>= 1;
18 }
19 return ans;
20 }
21 void prep(int n)
22 {
23     int i, j, w;
24     for(i = 0; i <= n; i++)
25     {
26         for(j = 1; j < (1 << i); j++)
27             rev[(1 << i) + j] = rev[(1 << (i - 1)) + (j >> 1)] | ((j & 1) << (i - 1));
28         w = qpow(3, (mod - 1) >> i);
29         omg[1 << i] = 1;
30         for(j = 1; j < (1 << i); j++)
31             omg[(1 << i) + j] = (ll)omg[(1 << i) + j - 1] * w % mod;
32     }
33 }
34 ull tmp[N * 2];
35 inline void ntt(int *A, int n, bool t) // 0 DFT 1 IDFT
36 {
37     int i, j, k, q, x;
38     for(i = 0; i < (1 << n); i++)
39         tmp[i] = A[i];
40     for(i = 0; i < (1 << n); i++)
41         if(rev[(1 << n) + i] > i)
42             swap(tmp[rev[(1 << n) + i]], tmp[i]);
43     for(i = 0; i < n; i++)
44     {
45         for(j = 0; j < (1 << n); j += (2 << i))
46             for(k = 0; k < (1 << i); k++)
47             {
48                 x = tmp[j + k + (1 << i)] * omg[(2 << i) + k] % mod;
49                 tmp[j + k + (1 << i)] = tmp[j + k] + mod - x;
50                 tmp[j + k] = tmp[j + k] + x;
51             }
52         if(i == 17 || i == n - 1)
53             for(j = 0; j < (1 << n); j++)
54                 tmp[j] %= mod;
55     }
56     for(i = 0; i < (1 << n); i++)
57         A[i] = tmp[i];
58     if(t)
59     {
60         for(i = 1; i < (1 << n); i++)
61             if(i < (1 << n) - i)
62                 swap(A[i], A[(1 << n) - i]);
63         q = qpow(1 << n, mod - 2);
64         for(i = 0; i < (1 << n); i++)
65             A[i] = (ll)A[i] * q % mod;
66     }
67 }

```

```

68 struct poly
69 {
70     vector<int> c;
71     poly()
72     {
73         c.clear();
74     }
75     poly(vector<int> d)
76     {
77         c = d;
78     }
79     friend poly operator * (poly f, poly g)
80     {
81         if(f.c.empty() || g.c.empty())
82             return poly();
83         poly h;
84         int i, l = 0;
85         while((f.c.size() + g.c.size()) >> l)
86             l++;
87         memset(X, 0, 4 << l);
88         memset(Y, 0, 4 << l);
89         for(i = 0; i < f.c.size(); i++)
90             X[i] = f.c[i];
91         for(i = 0; i < g.c.size(); i++)
92             Y[i] = g.c[i];
93         ntt(X, l, 0);
94         ntt(Y, l, 0);
95         for(i = 0; i < (1 << l); i++)
96             X[i] = (ll)X[i] * Y[i] % mod;
97         ntt(X, l, 1);
98         h.c.resize(f.c.size() + g.c.size() - 1);
99         for(i = 0; i < h.c.size(); i++)
100             h.c[i] = X[i];
101         return h;
102     }
103 };
104 vv solve(vi F, vv G, int n, int m) // [x^[0, n)y^(-m, 0]], [x^0]G = 1
105 {
106     int i, j, S, T, a, b, d = 2 * n - 1;
107     vv ans;
108     if(n == 1)
109     {
110         F.resize(m);
111         ans.resize(m);
112         for(i = 0; i < m; i++)
113             ans[i] = {F[i]};
114         return ans;
115     }
116     poly A, B, C;
117     vv H;
118     S = m * d + n;
119     A.c.resize(S);
120     B.c.resize(S);

```

```

121 for(i = 0; i <= m; i++)
122     for(j = 0; j < n; j++)
123     {
124         A.c[i * d + j] = G[i][j];
125         B.c[i * d + j] = (j & 1 ? mod - G[i][j] : G[i][j]);
126     }
127 A = A * B;
128 a = (n + 1) / 2;
129 b = 2 * m;
130 H.resize(b + 1);
131 for(i = 0; i <= b; i++)
132 {
133     H[i].resize(a);
134     for(j = 0; j < a; j++)
135         H[i][j] = A.c[i * d + 2 * j];
136 }
137 auto P = solve(F, H, a, b);
138 T = (b - 1) * d + n;
139 C.c.resize(T);
140 for(i = 0; i < b; i++)
141     for(j = 0; j < a; j++)
142         C.c[i * d + 2 * j] = P[b - 1 - i][j];
143 C = C * B;
144 ans.resize(m);
145 for(i = 0; i < m; i++)
146 {
147     ans[i].resize(n);
148     for(j = 0; j < n; j++)
149         ans[i][j] = C.c[(b - 1 - i) * d + j];
150 }
151 return ans;
152 }
153 poly comp(poly F, poly G, int n) // [x^0]G = 0
154 {
155     int i;
156     vv Gp;
157     Gp.resize(2);
158     Gp[0].resize(n);
159     Gp[1].resize(n);
160     Gp[0][0] = 1;
161     // 调用方给的 G 可能不足 n 个系数:必须补齐,否则下面 G.c[i] 越界读(ASAN 实测
162     // heap-buffer-overflow)。F 侧也一样,补到 n。
163     if((int) G.c.size() < n) G.c.resize(n);
164     if((int) F.c.size() < n) F.c.resize(n);
165     for(i = 1; i < n; i++)
166         Gp[1][i] = mod - G.c[i];
167     auto P = solve(F.c, Gp, n, 1);
168     return poly(P[0]);
169 }

```

6.3 多项式复合逆

模 998244353,给定 F ,求逆 G 使 $F(G(x)) = x \pmod{x^n}$ 。

- 完整程序(自带 main): 输入 n 与 F 的 n 个系数,输出 G 的 n 个系数
- comp_inv(F, n): 算法基于 Lagrange 反演 $[x^n] F^{-k}$,内部 pw_pj 做形式幂级数逆 + 幂迭代
- 要求: $[x^0] F = 0$ 且 $[x^1] F \neq 0$ (首项必须可逆,代码里直接求 qpow(F[1], mod-2))

坑: 与「多项式复合」同一套全局数组体系(数组大小约 $4.4e6+10$),内存大,搬用注意符号冲突;prep(22)除单位根外还预处理好阶乘/逆元表(fac/inv/Inv),整个程序只调用一次。

```

1 typedef vector<int> vi;
2 typedef vector<vi> vv;
3 typedef long long ll;
4 typedef unsigned long long ull;
5 const int N = 4.4e6 + 15, mod = 998244353;
6 int F[N], X[N], fac[N], inv[N], Inv[N];
7 int rev[N * 2], omg[N * 2];
8 int qpow(int x, int y)
9 {
10     int ans = 1;
11     while(y)
12     {
13         if(y & 1)
14             ans = (ll)ans * x % mod;
15         x = (ll)x * x % mod;
16         y >>= 1;
17     }
18     return ans;
19 }
20 void prep(int n)
21 {
22     int i, j, w, m = N - 5;
23     for(i = 0; i <= n; i++)
24     {
25         for(j = 1; j < (1 << i); j++)
26             rev[(1 << i) + j] = rev[(1 << (i - 1)) + (j >> 1)] | ((j & 1) << (i - 1));
27         w = qpow(3, (mod - 1) >> i);
28         omg[1 << i] = 1;
29         for(j = 1; j < (1 << i); j++)
30             omg[(1 << i) + j] = (ll)omg[(1 << i) + j - 1] * w % mod;
31     }
32     fac[0] = 1;
33     for(i = 1; i <= m; i++)
34         fac[i] = (ll)fac[i - 1] * i % mod;
35     inv[m] = qpow(fac[m], mod - 2);
36     for(i = m; i >= 1; i--)
37         inv[i - 1] = (ll)inv[i] * i % mod;
38     for(i = 1; i <= m; i++)
39         Inv[i] = (ll)inv[i] * fac[i - 1] % mod;
40 }
41 ull tmp[N * 2];
42 inline void ntt(int *A, int n, bool t) // 0 DFT 1 IDFT
43 {
44     int i, j, k, q, x;
45     for(i = 0; i < (1 << n); i++)

```

```

46     tmp[i] = A[i];
47     for(i = 0; i < (1 << n); i++)
48         if(rev[(1 << n) + i] > i)
49             swap(tmp[rev[(1 << n) + i]], tmp[i]);
50     for(i = 0; i < n; i++)
51     {
52         for(j = 0; j < (1 << n); j += (2 << i))
53             for(k = 0; k < (1 << i); k++)
54             {
55                 x = tmp[j + k + (1 << i)] * omg[(2 << i) + k] % mod;
56                 tmp[j + k + (1 << i)] = tmp[j + k] + mod - x;
57                 tmp[j + k] = tmp[j + k] + x;
58             }
59         if(i == 17 || i == n - 1)
60             for(j = 0; j < (1 << n); j++)
61                 tmp[j] %= mod;
62     }
63     for(i = 0; i < (1 << n); i++)
64         A[i] = tmp[i];
65     if(t)
66     {
67         for(i = 1; i < (1 << n); i++)
68             if(i < (1 << n) - i)
69                 swap(A[i], A[(1 << n) - i]);
70         q = qpow(1 << n, mod - 2);
71         for(i = 0; i < (1 << n); i++)
72             A[i] = (ll)A[i] * q % mod;
73     }
74 }
75 vi pv(vi F, int n, bool t)
76 {
77     int i;
78     F.resize(1 << n);
79     for(i = 0; i < (1 << n); i++)
80         X[i] = F[i];
81     ntt(X, n, t);
82     for(i = 0; i < (1 << n); i++)
83         F[i] = X[i];
84     return F;
85 }
86 struct poly
87 {
88     vector<int> c;
89     poly()
90     {
91         c.clear();
92     }
93     poly(vector<int> d)
94     {
95         c = d;
96     }
97     friend poly operator * (poly f, poly g)
98     {

```

```

99         if(f.c.empty() || g.c.empty())
100             return poly();
101         poly h;
102         int i, l = 0;
103         while((f.c.size() + g.c.size()) >> l)
104             l++;
105         f.c = pv(f.c, l, 0);
106         g.c = pv(g.c, l, 0);
107         for(i = 0; i < (1 << l); i++)
108             X[i] = (ll)f.c[i] * g.c[i] % mod;
109         ntt(X, l, 1);
110         h.c.resize(f.c.size() + g.c.size() - 1);
111         for(i = 0; i < h.c.size(); i++)
112             h.c[i] = X[i];
113         return h;
114     }
115 };
116 poly pw_pj(poly F, int n) // [x^{n-1}]1/(1-yF)
117 {
118     int i, j, S, R, m = 1, d = n - 1, l, r;
119     vv A, B;
120     vi pA, pB, pC;
121     A.resize(2);
122     A[0].resize(n);
123     A[0][0] = 1;
124     A[1].resize(n);
125     B = A;
126     for(i = 0; i < n; i++)
127         B[1][i] = mod - F.c[i];
128     while(d)
129     {
130         R = 2 * d + 1;
131         S = m * R + d + 1;
132         l = 0;
133         while((S - 1) >> l)
134             l++;
135         l++;
136         pA = pB = pC = {};
137         pA.resize(S);
138         pB.resize(S);
139         pC.resize(S);
140         for(i = 0; i <= m; i++)
141             for(j = 0; j <= d; j++)
142             {
143                 pA[i * R + j] = A[i][j];
144                 pB[i * R + j] = B[i][j];
145                 pC[i * R + j] = (j & 1 ? mod - B[i][j] : B[i][j]);
146             }
147         pA = pv(pA, l, 0);
148         pB = pv(pB, l, 0);
149         pC = pv(pC, l, 0);
150         for(i = 0; i < (1 << l); i++)
151         {

```

```

152     pA[i] = (ll)pA[i] * pC[i] % mod;
153     pB[i] = (ll)pB[i] * pC[i] % mod;
154 }
155 pA = pv(pA, l, 1);
156 pB = pv(pB, l, 1);
157 r = d & 1;
158 d /= 2;
159 m *= 2;
160 A.resize(m + 1);
161 B.resize(m + 1);
162 for(i = 0; i <= m; i++)
163 {
164     A[i].resize(d + 1);
165     B[i].resize(d + 1);
166     for(j = 0; j <= d; j++)
167     {
168         A[i][j] = pA[i * R + j * 2 + r];
169         B[i][j] = pB[i * R + j * 2];
170     }
171 }
172 }
173 F.c.resize(n); // 必须截到 n:否则调用方传进来的更多项式会把高次项留在后面,
174 // 接着 comp_inv 里的 reverse 会把它们倒到低次位置参与运算 → 静默算错
175 for(i = 0; i < n; i++)
176     F.c[i] = A[i][0];
177 return F;
178 }
179 // 要求 F.c.size() >= n(n == 1 时至少 2 个系数,内部会读 F.c[1])且 F[1] != 0;
180 // 返回长度恰为 n 的复合逆。
181 poly comp_inv(poly F, int n) // [x^n]F^k=k/n[x^{n-k}](G/x)^{-n}
182 {
183     // 补齐系数:下面要读 F.c[1](n == 1 时也得有),pw_pj 也要求 size >= n
184     if((int) F.c.size() < std::max(n, 2)) F.c.resize(std::max(n, 2));
185     int i, r, p = qpow(F.c[1], mod - 2), c;
186     for(i = 0, c = 1; i < n; i++, c = (ll)c * p % mod)
187         F.c[i] = (ll)F.c[i] * c % mod;
188     poly G;
189     F = pw_pj(F, n);
190     reverse(F.c.begin(), F.c.end());
191     for(i = 0; i < n; i++)
192         F.c[i] = (ll)F.c[i] * Inv[n - 1 - i] % mod * (n - 1) % mod;
193     r = mod - Inv[n - 1];
194     G.c = {1};
195     while(r)
196     {
197         if(r & 1)
198         {
199             G = G * F;
200             G.c.resize(n);
201         }
202         F = F * F;
203         F.c.resize(n);
204         r >>= 1;

```

```

205     }
206     G.c.resize(n);
207     for(i = n - 1; i > 0; i--)
208         G.c[i] = (ll)G.c[i - 1] * p % mod;
209     G.c[0] = 0;
210     return G;
211 }

```

6.4 最短线性递推

```

1 using poly = vector<mint>;
2 poly BM(const poly &a) {
3     int n = a.size();
4     if(!n) return poly();
5     int len = 0, off = 0; // len: 当前递推阶数;off: 距上次更新阶数的项数
6     mint s = 1; // s: 上次更新时的偏差
7     poly res(n + 1), lst(n + 1), c; // 开 n+1:len 最多到 n,末尾要用 res[len]
8     res[0] = 1;
9     For(i, 0, n - 1) {
10         mint d = 0; // 偏差 d = Σ res[j]*a[i-j](j <= len <=
11         i)
12         For(j, 0, len) d += res[j] * a[i - j];
13         ++off;
14         if(d.val() == 0) continue;
15         poly tmp = res;
16         mint k = d * s.inv(); // res ← res - (d/s) * lst * x^off
17         For(j, off, n) res[j] -= k * lst[j - off];
18         if(len * 2 > i) continue;
19         len = i - len + 1, lst = tmp, s = d, off = 0;
20     }
21     c.resize(len);
22     For(i, 0, len - 1) c[i] = -res[i + 1]; // res[0]=1 且 d=0 ⇒ a[i] = -Σ_{j>=1}
23     res[j]a[i-j]
24     return c;
25 }

```

6.5 线性递推第 n 项

```

1 using poly = vector<mint>;
2 mint bostan_mori(poly P, poly Q, ll n) {
3     // 平凡情形:Q 是常数(deg Q = 0)时 P/Q 只有第 0 项非零。
4     // 另外 mul() 对长度 1 的输入会返回空向量,所以这里必须先短路。
5     if((int) Q.size() <= 1) return n == 0 ? (P.empty() ? mint(0) : P[0]) / Q[0] :
6     mint(0);
7     // 高于 deg Q - 1 的 P 项对第 n 项没有影响,直接截掉(不截会让卷积越算越大)
8     if((int) P.size() > (int) Q.size() - 1) P.resize(Q.size() - 1);
9     while (n) {
10         poly Qm = Q;
11         For(i, 1, (int) Qm.size() - 1) if(i & 1) Qm[i] = -Qm[i]; // Q(-x)
12         poly U = mul(P, Qm), V = mul(Q, Qm);
13         P.clear();

```

```

13     for(int i = n & 1; i < (int) U.size(); i += 2) P.push_back(U[i]); // 取偶数/奇
    数项
14     Q.clear();
15     for(int i = 0; i < (int) V.size(); i += 2) Q.push_back(V[i]);
16     n >>= 1;
17 }
18 return P[0] / Q[0];
19 }

```

6.6 复数 FFT 卷积

```

1 using cp = complex<db>;
2 void fft(vect<cp> &a, int k) {
3     int n = a.size();
4     static vect<int> r;
5     if((int) r.size() != n) {
6         int l = __lg(n);
7         r.resize(n);
8         For(i, 0, n - 1) r[i] = (r[i >> 1] >> 1) | ((i & 1) << (l - 1));
9     }
10    For(i, 0, n - 1) if(i < r[i]) swap(a[i], a[r[i]]);
11    for(int m = 1; m < n; m <= 1) {
12        cp wm = polar((db) 1, acos((db) -1) / m * k);
13        for(int i = 0; i < n; i += m + m) {
14            cp w = 1;
15            For(j, 0, m - 1) {
16                cp x = a[i + j], y = a[i + j + m] * w;
17                a[i + j] = x + y, a[i + j + m] = x - y, w *= wm;
18            }
19        }
20    }
21    if(k == -1) For(i, 0, n - 1) a[i] /= n;
22 }
23 vect<db> fft_conv(const vect<db> &a, const vect<db> &b) {
24     int n = a.size(), m = b.size();
25     if(!n || !m) return {};
26     int l = 1;
27     while (l < n + m - 1) l <= 1;
28     vect<cp> fa(l), fb(l);
29     For(i, 0, n - 1) fa[i] = a[i];
30     For(i, 0, m - 1) fb[i] = b[i];
31     fft(fa, 1), fft(fb, 1);
32     For(i, 0, l - 1) fa[i] *= fb[i];
33     fft(fa, -1);
34     vect<db> res(n + m - 1);
35     For(i, 0, n + m - 2) res[i] = fa[i].real();
36     return res;
37 }

```

6.7 多点求值

```

1 using poly = vector<mint>;

```

```

2 static void _eval_build(poly *t, int k, int l, int r, const poly &xs) {
3     if(l == r) return void(t[k] = poly{-xs[l], mint(1)}); // x - xs[l]
4     int mid = (l + r) >> 1;
5     _eval_build(t, k + k, l, mid, xs), _eval_build(t, k + k + 1, mid + 1, r, xs);
6     t[k] = mul(t[k + k], t[k + k + 1]);
7 }
8 static void _eval_solve(const poly *t, int k, int l, int r, const poly &f, poly
    &res) {
9     poly g = poly_mod(f, t[k]);
10    if(l == r) return void(res[l] = g.empty() ? mint() : g[0]);
11    int mid = (l + r) >> 1;
12    _eval_solve(t, k + k, l, mid, g, res), _eval_solve(t, k + k + 1, mid + 1, r, g,
    res);
13 }
14 poly poly_eval(const poly &f, const poly &xs) {
15     int m = xs.size();
16     if(!m) return poly();
17     vector<poly> t(4 * m);
18     _eval_build(t.data(), 1, 0, m - 1, xs);
19     poly res(m);
20     _eval_solve(t.data(), 1, 0, m - 1, f, res);
21     return res;
22 }

```

6.8 多项式指数

```

1 using poly = vector<mint>;
2 poly poly_exp(const poly &f, int n) {
3     poly g{1};
4     for(int m = 1; m < n; m <= 1) {
5         int t = min(m << 1, n);
6         poly c = poly_ln(g, t); // ln g(mod x^t), g 只有 m 项、高次按 0 参与运算
7         For(i, 0, t - 1) c[i] = (i < (int) f.size() ? f[i] : mint()) - c[i];
8         c[0] += 1; // 1 - ln g + f
9         g = mul(g, c);
10        g.resize(t);
11    }
12    g.resize(n);
13    return g;
14 }

```

6.9 多项式对数

```

1 using poly = vector<mint>;
2 static const poly &ln_inv_num(int n) { // 1..n 的模逆元表(递推 inv[i] = -(M/
    i)*inv[M%i])
3     static poly iv{0, 1};
4     if((int) iv.size() <= n) {
5         int m = iv.size();
6         iv.resize(n + 1);
7         For(i, m, n) iv[i] = -mint(mint::getM() / i) * iv[mint::getM() % i];
8     }

```

```

9     return iv;
10 }
11 poly poly_ln(const poly &f, int n) {
12     if(n <= 0) return poly();
13     poly d(n - 1);
14     For(i, 1, min((int) f.size(), n) - 1) d[i - 1] = f[i] * i; // f'
15     d = mul(d, poly_inv(f, n)); // f'/f
16     d.resize(n - 1);
17     const poly &iv = _ln_inv_num(n);
18     poly g(n);
19     For(i, 1, n - 1) g[i] = d[i - 1] * iv[i]; // 积分:系数除以 i
20     return g;
21 }

```

6.10 多项式开根

```

1 using poly = vector<mint>;
2 poly poly_sqrt(const poly &f, int n) {
3     static const mint ih = mint(2).inv();
4     poly g{1};
5     for(int m = 1; m < n; m <= 1) {
6         int t = min(m << 1, n);
7         poly c(f.begin(), f.begin() + min((int) f.size(), t)); // f mod x^t
8         poly d = mul(c, poly_inv(g, t)); // f/g
9         d.resize(t);
10        g.resize(t);
11        For(i, 0, t - 1) g[i] = (g[i] + d[i]) * ih; // (g + f/g)/2
12    }
13    g.resize(n);
14    return g;
15 }

```

6.11 多项式求逆

```

1 using poly = vector<mint>;
2 poly poly_inv(const poly &f, int n) {
3     poly g{f[0].inv()};
4     for(int m = 1; m < n; m <= 1) {
5         int t = min(m << 1, n);
6         poly c(f.begin(), f.begin() + min((int) f.size(), t)); // f mod x^t
7         poly d = mul(g, c);
8         d.resize(t);
9         For(i, 0, t - 1) d[i] = (i == 0 ? mint(2) : mint()) - d[i]; // 2 - f*g
10        g = mul(g, d);
11        g.resize(t);
12    }
13    g.resize(n);
14    return g;
15 }

```

6.12 带余除法

```

1 using poly = vector<mint>;
2 pair<poly, poly> poly_divmod(const poly &f, const poly &g) {
3     int n = f.size(), m = g.size();
4     if(n < m) return {poly(), f};
5     int k = n - m + 1;
6     poly rf(f.rbegin(), f.rend()), rg(g.rbegin(), g.rend());
7     poly q = mul(rf, poly_inv(rg, k));
8     q.resize(k);
9     reverse(q.begin(), q.end());
10    poly r = mul(q, g);
11    r.resize(n);
12    For(i, 0, n - 1) r[i] = f[i] - r[i];
13    r.resize(m - 1);
14    return {q, r};
15 }
16 poly poly_mod(const poly &f, const poly &g) { return poly_divmod(f, g).second; }

```

6.13 快速插值

```

1 using poly = vector<mint>;
2 static void _interp_build(poly *t, int k, int l, int r, const poly &xs) {
3     if(l == r) return void(t[k] = poly{-xs[l], mint(1)}); // x - xs[l]
4     int mid = (l + r) >> 1;
5     _interp_build(t, k + k, l, mid, xs), _interp_build(t, k + k + 1, mid + 1, r,
6     xs);
7     t[k] = mul(t[k + k], t[k + k + 1]);
8 }
9 // P_node = P_left * t[right] + P_right * t[left]
10 static poly _interp_rec(const poly *t, int k, int l, int r, const poly &w) {
11     if(l == r) return poly{w[l]};
12     int mid = (l + r) >> 1;
13     poly a = mul(_interp_rec(t, k + k, l, mid, w), t[k + k + 1]);
14     poly b = mul(_interp_rec(t, k + k + 1, mid + 1, r, w), t[k + k]);
15     a.resize(max(a.size(), b.size()), mint(0));
16     For(i, 0, (int) b.size() - 1) a[i] += b[i];
17     return a;
18 }
19 poly poly_interp(const poly &xs, const poly &ys) {
20     int m = xs.size();
21     if(!m) return poly();
22     vector<poly> t(4 * m);
23     _interp_build(t.data(), 1, 0, m - 1, xs);
24     poly dM(m);
25     For(i, 1, m) dM[i - 1] = t[1][i] * i; // M' 的系数
26     poly ev = poly_eval(dM, xs), w(m);
27     For(i, 0, m - 1) w[i] = ys[i] / ev[i]; // M'(xs[i]) 互异时非 0
28     poly res = _interp_rec(t.data(), 1, 0, m - 1, w);
29     res.resize(m);
30     return res;
31 }

```

7 计算几何

namespace Geo,eps=1e-10,双精度;p2 为点,seg 为线段。

- 基本: sign/cmp,det(叉积)、点乘 *、r90/rot/unit、dis/dis2
- 线段: chkss(相交含端点)/chkss_s(严格)、isll(直线交)、ons/ons_s(点在线上)、proj/reflect/nearest/diss(投影/翻转/点线距/线线距)
- 多边形: area(面积,有符号)、contain(点形关系: 0 外/1 线上/2 内)、convex_hull(凸包,默认去共线点,nos=1 保留)、convex_diameter(凸包直径)、convex_cut(半平面裁剪)
- 圆: circumcenter(外心)、circumcircle_diameter(外接圆直径,三点共线返 -1)

坑: operator-= 和 operator/= 实现成了 $x = x + y$ (笔误!)——需要减/除时手写;area 输出带符号(按顶点序),要面积就取 abs;convex_hull 会改写输入数组,保序请另存;eps=1e-10 对边界情况(多点共线等)需要留心。

```
1 namespace Geo {
2
3 const db eps = 1e-10;
4
5 struct p2;
6 struct seg;
7 int sign(db x) { return x < -eps ? -1 : x > eps; }
8 int cmp(db x, db y) { return sign(x - y); }
9 const db pi = acos(db(-1));
10 struct p2 {
11     db x, y;
12     db det(const p2 &b) const { return x * b.y - y * b.x; }
13     db alpha() const { return atan2(y, x); }
14     bool operator==(const p2 &b) const {
15         return !cmp(x, b.x) && !cmp(y, b.y);
16     }
17     bool operator<(const p2 &b) const {
18         int c = cmp(x, b.x);
19         if(c) return !~c;
20         return !~cmp(y, b.y);
21     }
22 };
23 p2 r90(p2 x) { return {-x.y, x.x}; }
24 p2 rot(p2 x, db a) { return {x.x * cos(a) - x.y * sin(a), x.x * sin(a) + x.y * cos(a)}; }
25 p2 operator+(p2 x, p2 y) { return {x.x + y.x, x.y + y.y}; }
26 p2 operator-(p2 x, p2 y) { return {x.x - y.x, x.y - y.y}; }
27 p2 operator/(p2 x, db y) { return {x.x / y, x.y / y}; }
28 p2 operator*(p2 x, db y) { return {x.x * y, x.y * y}; }
29 p2 operator*(db y, p2 x) { return {x.x * y, x.y * y}; }
30 db operator*(p2 x, p2 y) { return x.x * y.x + x.y * y.y; }
31 p2 operator+=(p2 &x, p2 y) { return x = x + y; }
32 p2 operator-=(p2 &x, p2 y) { return x = x - y; }
33 p2 operator/=(p2 &x, db y) { return x = x / y; } // 原来是 (p2&, p2) 且写成加法:连 a /=
    2.0 都编译不过
34 db dis2(p2 x) { return x.x * x.x + x.y * x.y; }
35 db dis(p2 x) { return hypot(x.x, x.y); }
```

```
36 p2 unit(p2 x) { return x / dis(x); }
37 struct seg {
38     p2 x, y;
39     p2 dir() const { return y - x; }
40 };
41 db cross(const seg &p, const p2 &q) { return p.dir().det(q - p.x); }
42 int crossop(const seg &p, const p2 &q) { return sign(cross(p, q)); }
43 bool inc(seg l, const p2 &b) { return !crossop(l, b); }
44 bool eqll(const seg &a, const seg &b) {
45     return inc(a, b.x) && inc(a, b.y);
46 }
47 bool chkll(const seg &a, const seg &b) {
48     db a1 = cross(b, a.x), a2 = -cross(b, a.y);
49     return !!sign(a1 + a2);
50 }
51 p2 isll(const seg &a, const seg &b) {
52     db a1 = cross(b, a.x), a2 = -cross(b, a.y);
53     return (a.x * a2 + a.y * a1) / (a1 + a2);
54 }
55 bool chkss(const seg &p, const seg &q) {
56     auto is = [] (db l1, db r1, db l2, db r2) {
57         if(l1 > r1) swap(l1, r1);
58         if(l2 > r2) swap(l2, r2);
59         return !(cmp(r1, l2) == -1 || cmp(r2, l1) == -1);
60     };
61     p2 p1 = p.x, p2 = p.y, q1 = q.x, q2 = q.y;
62     return is(p1.x, p2.x, q1.x, q2.x) && is(p1.y, p2.y, q1.y, q2.y) &&
63         crossop(p, q1) * crossop(p, q2) <= 0 && crossop(q, p1) * crossop(q, p2)
64         <= 0;
65 }
66 bool chkss_s(const seg &p, const seg &q) {
67     p2 p1 = p.x, p2 = p.y, q1 = q.x, q2 = q.y;
68     return crossop(p, q1) * crossop(p, q2) < 0 && crossop(q, p1) * crossop(q, p2) <
69         0;
70 }
71 bool isMid(const db &a, const db &m, const db &b) { return !cmp(a, b) || !cmp(b, m)
    || cmp(a, m) != cmp(b, m); }
72 bool isMid(const p2 &a, const p2 &m, const p2 &b) { return isMid(a.x, m.x, b.x) &&
    isMid(a.y, m.y, b.y); }
73 bool ons(const seg &p, const p2 &q) { return !crossop(p, q) && isMid(p.x, q, p.y); }
74 bool ons_s(const seg &p, const p2 &q) {
75     return !crossop(p, q) && sign((q - p.x) * (p.x - p.y)) * sign((q - p.y) * (p.x -
    p.y)) < 0;
76 }
77 p2 proj(const seg &p, const p2 &q) {
78     p2 dir = p.y - p.x;
79     return p.x + dir * ((dir * (q - p.x)) / dis2(dir));
80 }
81 p2 reflect(const seg &p, const p2 &q) {
82     return proj(p, q) * 2 - q;
83 }
84 db nearest(const seg &p, const p2 &q) {
85     p2 h = proj(p, q);
```

```

84     if(isMid(p.x, h, p.y)) return dis2(q - h);
85     return min(dis2(p.x - q), dis2(p.y - q));
86 }
87 db disss(const seg &p, const seg &q) {
88     if(chkss(p, q)) return 0;
89     return min({nearest(p, q.x), nearest(p, q.y), nearest(q, p.x), nearest(q,
    p.y)});
90 }
91 db area(int n, p2 *a) {
92     db ret = 0;
93     For(i, 0, n - 1) ret += a[i].det(a[(i + 1) % n]);
94     return ret / 2;
95 }
96 int contain(int n, p2 *a, const p2 &p) { // 0: outside, 1: on seg, 2: inside
97     int ret = 0;
98     For(i, 0, n - 1) {
99         p2 u = a[i], v = a[(i + 1) % n];
100         if(ons({u, v}, p)) return 1;
101         if(cmp(u.y, v.y) <= 0) swap(u, v);
102         if(cmp(p.y, u.y) > 0 || cmp(p.y, v.y) <= 0) continue;
103         ret ^= crossop({p, u}, v) > 0;
104     }
105     return ret * 2;
106 }
107 // 求凸包,返回点数,k 个顶点按逆时针存在 b[0..k-1]。nos=0 去掉共线点,nos=1 保留。
108 // **前置条件:输出缓冲 b 至少要能放 2n + 2 个点**(不是 n 个!扫下凸壳时会先写满 n 个位置,
109 // 之后上凸壳还会再写;实测 ASAN:p2 b[5] + 点 (i,i^2) 在写 b[5] 时 stack-buffer-overflow)。
110 // 输入 a 会被原地排序;b 与 a 不能是同一块内存。
111 int convex_hull(int n, p2 *a, p2 *b, bool nos = 0) {
112     if(n <= 1) return 1;
113     sort(a, a + n);
114     int k = 0;
115     For(i, 0, n - 1) {
116         while(k > 1 && crossop({b[k - 2], b[k - 1]}, a[i]) <= -nos) --k;
117         b[k++] = a[i];
118     }
119     int t = k;
120     rFor(i, n - 2, 0) {
121         while(k > t && crossop({b[k - 2], b[k - 1]}, a[i]) <= -nos) --k;
122         if(i) b[k++] = a[i];
123     }
124     return k;
125 }
126 db convex_diameter(int n, p2 *a) {
127     if(n <= 1) return 0;
128     int is = 0, js = 0;
129     For(k, 1, n - 1) is = a[k] < a[is] ? k : is, js = a[js] < a[k] ? k : js;
130     int i = is, j = js;
131     db ret = dis2(a[i] - a[j]);
132     do {
133         if((a[(i + 1) % n] - a[i]).det(a[(j + 1) % n] - a[j]) >= 0)
134             (++j) %= n;
135     } else

```

```

136         (++i) %= n;
137         ret = max(ret, dis2(a[i] - a[j]));
138     } while(i != is || j != js);
139     return ret;
140 }
141 vector<p2> convex_cut(int n, p2 *a, const seg &q) {
142     vector<p2> qs;
143     For(i, 0, n - 1) {
144         p2 p1 = a[i], p2 = a[(i + 1) % n];
145         int d1 = crossop(q, p1), d2 = crossop(q, p2);
146         if(d1 >= 0) qs.push_back(p1);
147         if(d1 * d2 < 0) qs.push_back(isll({p1, p2}, q));
148     }
149     return qs;
150 }
151 db circumcircle_diameter(p2 x, p2 y, p2 z) {
152     if(!crossop({x, y}, z)) return -1;
153     return dis(x - y) * dis(y - z) * dis(z - x) / abs((y - x).det(z - x));
154 }
155 array<db, 2> circumcenter(p2 x, p2 y, p2 z) {
156     auto [x1, y1] = x;
157     auto [x2, y2] = y;
158     auto [x3, y3] = z;
159     auto A = dis2(x), B = dis2(y), C = dis2(z);
160     auto u1 = x1 - x2, u2 = x1 - x3, v1 = y1 - y2, v2 = y1 - y3;
161     return {db((C - A) * v1 - (B - A) * v2) / (2 * u1 * v2 - 2 * u2 * v1), db((C -
    A) * u1 - (B - A) * u2) / (2 * v1 * u2 - 2 * v2 * u1)};
162 }
163
164 } // namespace Geo
165 using namespace Geo;

```

7.1 Delaunay 三角剖分

```

1 namespace Delaunay {
2
3 vect<p2> pts; // 去重后的点,按 (x, y) 精确升序
4 struct Tri { int v[3], f[3], ver; }; // v 逆时针;f[i] 是顶点 i 对边的邻接三角形(-1 表示
    凸包边)
5 vector<Tri> tri;
6 vect<char> live;
7 vect<int> fre;
8 int verc;
9
10 int nw() {
11     if(!fre.empty()) {
12         int t = fre.back();
13         fre.pop_back(), live[t] = 1;
14         return t;
15     }
16     tri.push_back(Tri()), live += 1;
17     return (int) tri.size() - 1;

```



```

18 }
19 void rm(int t) { live[t] = 0, fre += t; }
20 bool lt(p2 u, p2 v) { return u.x != v.x ? u.x < v.x : u.y < v.y; }
21 int ori(int a, int b, int c) { return sign((pts[b] - pts[a]).det(pts[c] - pts[a])); }
22 int ill(int p, int a, int b, int c) { // p 是否严格落在逆时针三角形 (a,b,c) 的外接圆内
23     p2 A = pts[a] - pts[p], B = pts[b] - pts[p], C = pts[c] - pts[p];
24     db t1 = dis2(A) * B.det(C), t2 = dis2(B) * C.det(A), t3 = dis2(C) * A.det(B);
25     return t1 + t2 + t3 > 1e-12L * (abs(t1) + abs(t2) + abs(t3));
26 }
27 int finde(int t, int a, int b) { // 三角形 t 中「有序边 (a,b)」的编号
28     ForD(i, 0, 3) if(tri[t].v[(i + 1) % 3] == a && tri[t].v[(i + 2) % 3] == b)
29         return i;
30     return -1;
31 void lnk(int t, int i, int g) { // 把 t 的第 i 条边和 g 连起来(双向); g = -1 表示凸包边
32     tri[t].f[i] = g;
33     if(g >= 0) tri[g].f[finde(g, tri[t].v[(i + 2) % 3], tri[t].v[(i + 1) % 3])] = t;
34 }
35 int mk(int a, int b, int c) { // 新建逆时针三角形 (a,b,c)
36     if(ori(a, b, c) < 0) swap(b, c);
37     int t = nw();
38     tri[t].v[0] = a, tri[t].v[1] = b, tri[t].v[2] = c;
39     tri[t].f[0] = tri[t].f[1] = tri[t].f[2] = -1, tri[t].ver = ++verc;
40     return t;
41 }
42 typedef array<int, 3> Tst; // 待检查的边:(三角形, 该边对面顶点在三角形里的下标, 三角形版本号)
43 void flipall(vector<Tst> &st) { // 反复翻非法边,直到栈空(凸包边永不翻)
44     while(!st.empty()) {
45         Tst e = st.back();
46         st.pop_back();
47         int t = e[0], i = e[1];
48         if(!live[t] || tri[t].ver != e[2]) continue; // 这条边早已被别的翻边改掉了
49         int a = tri[t].v[(i + 1) % 3], b = tri[t].v[(i + 2) % 3], g = tri[t].f[i];
50         if(g < 0) continue; // 凸包边:合法
51         int k = finde(g, b, a), q = tri[g].v[k];
52         if(!ill(q, tri[t].v[0], tri[t].v[1], tri[t].v[2])) continue; // 合法(含共
圆):不翻
53         int p = tri[t].v[i];
54         int X = tri[t].f[(i + 1) % 3], Y = tri[t].f[(i + 2) % 3];
55         int W = tri[g].f[(k + 1) % 3], Z = tri[g].f[(k + 2) % 3];
56         tri[t].v[0] = p, tri[t].v[1] = a, tri[t].v[2] = q; // (p,a,b)+(b,a,q) ->
(p,a,q)+(p,q,b)
57         tri[g].v[0] = p, tri[g].v[1] = q, tri[g].v[2] = b;
58         lnk(t, 0, W), lnk(t, 2, Y), lnk(t, 1, g);
59         lnk(g, 0, Z), lnk(g, 1, X);
60         tri[t].ver = ++verc, tri[g].ver = ++verc;
61         st.push_back({t, 0, tri[t].ver}), st.push_back({t, 2, tri[t].ver});
62         st.push_back({g, 0, tri[g].ver}), st.push_back({g, 1, tri[g].ver});
63     }
64 }
65

```

```

66 vect<p2> delaunay(vect<p2> a) {
67     pts.clear(), tri.clear(), live.clear(), fre.clear(), verc = 0;
68     sort(a.begin(), a.end(), lt);
69     ForD(i, 0, (int) a.size())
70         if(pts.empty() || pts.back().x != a[i].x || pts.back().y != a[i].y) pts +=
a[i];
71     int n = (int) pts.size();
72     if(n < 3) return {};
73     // 注意 convex_hull 会往 b 里写下凸壳 + 上凸壳,缓冲必须比 n 大(实测就要 n+1 个位置,
74     // 理论 upper 2n-2,而它的注释里没写这条前置条件),所以这里一律给 2n+2。
75     vect<p2> q(pts.begin(), pts.end()), hb(2 * n + 2);
76     int k = convex_hull(n, q.data(), hb.data()); // 凸包(逆时针,去掉共线点)
77     if(k < 3) return {}; // 全部共线:没有任何非退化三角形
78     vect<int> hv(k), fs(k - 2);
79     ForD(j, 0, k) hv[j] = (int) (lower_bound(pts.begin(), pts.end(), hb[j], lt) -
pts.begin());
80     vector<Tst> st;
81     For(i, 1, k - 2) fs[i - 1] = mk(hv[0], hv[i], hv[i + 1]); // 先做扇形剖分
82     For(i, 1, k - 3) lnk(fs[i - 1], 1, fs[i]);
83     ForD(t, 0, (int) tri.size()) ForD(i, 0, 3) st.push_back({t, i, tri[t].ver}); //
把它翻成 Delaunay
84     flipall(st);
85     vect<int> in; // 内部点(不在凸包上的点;可能正好落在凸包边上)
86     vect<char> onh(n, 0);
87     ForD(j, 0, k) onh[hv[j]] = 1;
88     ForD(i, 0, n) if(!onh[i]) in += i;
89     mt19937 rng(20240915);
90     shuffle(in.begin(), in.end(), rng);
91     int last = fs[0];
92     ForD(i, 0, (int) in.size()) {
93         int p = in[i], cur = last, go;
94         for(;;) { // 游走定位:p 一定在凸包内(或凸包边上),所以走不出三角剖分
95             go = -1;
96             ForD(j, 0, 3) if(ori(tri[cur].v[(j + 1) % 3], tri[cur].v[(j + 2) % 3],
p) < 0) { go = j; break; }
97             if(go < 0 || tri[cur].f[go] < 0) break;
98             cur = tri[cur].f[go];
99         }
100         int e = -1; // p 正好落在某条边上
101         ForD(j, 0, 3) if(!ori(tri[cur].v[(j + 1) % 3], tri[cur].v[(j + 2) % 3], p))
{ e = j; break; }
102         st.clear();
103         if(e < 0) { // 1. p 在三角形内:拆成 3 个
104             int a = tri[cur].v[0], b = tri[cur].v[1], c = tri[cur].v[2];
105             int f0 = tri[cur].f[0], f1 = tri[cur].f[1], f2 = tri[cur].f[2];
106             rm(cur);
107             int t1 = mk(a, b, p), t2 = mk(b, c, p), t3 = mk(c, a, p);
108             lnk(t1, 2, f2), lnk(t2, 2, f0), lnk(t3, 2, f1);
109             lnk(t1, 0, t2), lnk(t2, 0, t3), lnk(t3, 0, t1);
110             st.push_back({t1, 2, tri[t1].ver}), st.push_back({t2, 2, tri[t2].ver}),
st.push_back({t3, 2, tri[t3].ver});
111             last = t1;
112         } else { // 2. p 在边 (a,b) 上:两侧的三角形各拆成 2 个(凸包边只有一侧)

```

```

113     int a = tri[cur].v[(e + 1) % 3], b = tri[cur].v[(e + 2) % 3], c =
    tri[cur].v[e];
114     int g = tri[cur].f[e];
115     int fc1 = tri[cur].f[(e + 1) % 3], fc2 = tri[cur].f[(e + 2) % 3];
116     if(g < 0) { // 凸包边: 只有一个三角形, 拆成 2 个
117         rm(cur);
118         int t1 = mk(c, a, p), t2 = mk(c, p, b);
119         lnk(t1, 1, t2);
120         lnk(t1, 2, fc2), lnk(t2, 1, fc1);
121         st.push_back({t1, 2, tri[t1].ver}), st.push_back({t2, 1,
    tri[t2].ver});
122         last = t1;
123     } else {
124         int kk = finde(g, b, a), d = tri[g].v[kk];
125         int fn1 = tri[g].f[(kk + 1) % 3], fn2 = tri[g].f[(kk + 2) % 3];
126         rm(cur), rm(g);
127         int t1 = mk(c, a, p), t2 = mk(c, p, b), u1 = mk(d, b, p), u2 = mk(d,
    p, a);
128         lnk(t1, 0, u2), lnk(t1, 1, t2), lnk(t2, 0, u1), lnk(u1, 1, u2);
129         lnk(t1, 2, fc2), lnk(t2, 1, fc1), lnk(u1, 2, fn2), lnk(u2, 1, fn1);
130         st.push_back({t1, 2, tri[t1].ver}), st.push_back({t2, 1,
    tri[t2].ver});
131         st.push_back({u1, 2, tri[u1].ver}), st.push_back({u2, 1,
    tri[u2].ver});
132         last = t1;
133     }
134 }
135 flipall(st);
136 }
137 vect<p2> res;
138 ForD(t, 0, (int) tri.size()) if(live[t]) {
139     int a = tri[t].v[0], b = tri[t].v[1], c = tri[t].v[2];
140     res += pts[a], res += pts[b], res += pts[c];
141 }
142 return res;
143 }
144
145 } // namespace Delaunay
146 using namespace Delaunay;

```

7.2 Voronoi 图

```

1 namespace Voronoi {
2
3 array<p2, 2> box; // 裁剪用的包围盒: box[0] 左下、box[1] 右上
4
5 // 到 a 比到 b 近的半平面: 以 ab 的中垂线为界, 方向取 r90(b - a), 这样 a 在它的左侧
6 // (convex_cut 保留 cross(q, ·) >= 0 的一侧, 正好是 a 这一侧)。
7 seg bisector(p2 a, p2 b) {
8     p2 m = (a + b) / 2;
9     return {m, m + r90(b - a)};
10 }
11

```

```

12 vect<vect<p2>> voronoi(vect<p2> a) {
13     int n = (int) a.size();
14     vect<vect<p2>> res;
15     if(!n) return res;
16     db x1 = a[0].x, x2 = a[0].x, y1 = a[0].y, y2 = a[0].y;
17     ForD(i, 1, n) x1 = min(x1, a[i].x), x2 = max(x2, a[i].x), y1 = min(y1, a[i].y),
    y2 = max(y2, a[i].y);
18     db r = max(x2 - x1, y2 - y1) + 1;
19     box = {p2{x1 - r, y1 - r}, p2{x2 + r, y2 + r}};
20     vect<p2> ini;
21     ini += p2{box[0].x, box[0].y}, ini += p2{box[1].x, box[0].y};
22     ini += p2{box[1].x, box[1].y}, ini += p2{box[0].x, box[1].y}; // 逆时针
23     ForD(i, 0, n) {
24         vect<p2> cur = ini;
25         ForD(j, 0, n) {
26             if(j == i || a[i] == a[j] || cur.size() < 3) continue;
27             cur = convex_cut((int) cur.size(), cur.data(), bisector(a[i], a[j]));
28         }
29         res += cur;
30     }
31     return res;
32 }
33
34 } // namespace Voronoi
35 using namespace Voronoi;

```

7.3 三维凸包

```

1 namespace Geo {
2
3 struct face3 {
4     int a, b, c;
5 };
6
7 vector<face3> convex3d(const vector<p3> &a) {
8     int n = a.size();
9     vector<face3> ret;
10    if(n < 4) return ret;
11    // ---- 1. 找一个不共面的初始四面体 A,B,C,D ----
12    int A = 0, B = -1, C = -1, D = -1;
13    ForD(i, 0, n) if(sign(dis(a[i] - a[A]))) { B = i; break; } // 与 A 不重合
14    if(B < 0) return ret; // 全部点重合
15    p3 w0 = a[B] - a[A];
16    ForD(i, 0, n) if(sign(dis(cross(a[i] - a[A], w0)) / dis(w0))) { C = i;
    break; } // 不共线
17    if(C < 0) return
    ret; // 全部点共线
18    plane p0(a[A], a[B], a[C]);
19    ForD(i, 0, n) if(sign(abs(p0.side(a[i])))) { D = i; break; } // 离开 A,B,C 所在平
    面
20    if(D < 0) return ret; // 全部点共面
21    // ---- 2. 把四个种子点排到前面, 后面按顺序插入 ----
22    vector<int> id; // 局部下标 -> 原下标

```

```

23 id.push_back(A), id.push_back(B), id.push_back(C), id.push_back(D);
24 ForD(i, 0, n) if(i != A && i != B && i != C && i != D) id.push_back(i);
25 vector<p3> b(n);
26 ForD(i, 0, n) b[i] = a[id[i]];
27 struct F {
28     int a, b, c;
29     plane p;
30 };
31 vector<F> f;
32 f.push_back(F{0, 1, 2, plane(b[0], b[1], b[2])}); // 两个朝向的初始三角形,
33 f.push_back(F{0, 2, 1, plane(b[0], b[2], b[1])}); // 插入第 4 个点时朝向会自动定好
34 unordered_map<ll, int> vis; // 有向边 (u,v) -> 最后一次被删的
    轮次
35 For(i, 3, n - 1) {
36     vector<F> keep, kill;
37     ForD(t, 0, f.size()) {
38         if(f[t].p.side(b[i]) > eps) kill.push_back(f[t]); // 严格在外侧 -> 可见,删
    掉重连
39         else keep.push_back(f[t]);
40     }
41     if(kill.empty()) continue; // 点在凸包内(或离面不超过 eps):不产生新顶点
42     int e[3][2];
43     ForD(t, 0, kill.size()) { // 先把被删面的三条有向边打上本轮标记
44         e[0][0] = kill[t].a, e[0][1] = kill[t].b;
45         e[1][0] = kill[t].b, e[1][1] = kill[t].c;
46         e[2][0] = kill[t].c, e[2][1] = kill[t].a;
47         ForD(k, 0, 3) vis[(ll) e[k][0] * n + e[k][1]] = i;
48     }
49     ForD(t, 0, kill.size()) { // 反向边没被标记的边就是地平线,与新点连成新面
50         e[0][0] = kill[t].a, e[0][1] = kill[t].b;
51         e[1][0] = kill[t].b, e[1][1] = kill[t].c;
52         e[2][0] = kill[t].c, e[2][1] = kill[t].a;
53         ForD(k, 0, 3)
54             if(vis[(ll) e[k][0] * n + e[k][1]] == i && vis[(ll) e[k][1] * n +
e[k][0]] != i)
55             keep.push_back(F{e[k][0], e[k][1], i, plane(b[e[k][0]], b[e[k]
[1]], b[i])});
56     }
57     f.swap(keep);
58 }
59 ForD(t, 0, f.size()) ret.push_back(face3{id[f[t].a], id[f[t].b], id[f[t].c]});
60 return ret;
61 }
62
63 // 凸包表面积(f 的面法向朝外,这里只用三角形面积,与朝向无关)
64 db hull_area(const vector<p3> &a, const vector<face3> &f) {
65     db s = 0;
66     ForD(t, 0, f.size()) s += area2(a[f[t].a], a[f[t].b], a[f[t].c]);
67     return s / 2;
68 }
69 // 凸包体积:每个面与原点围成的四面体带号体积求和(朝向一致时符号相同,取绝对值)
70 db hull_volume(const vector<p3> &a, const vector<face3> &f) {
71     db s = 0;

```

```

72 ForD(t, 0, f.size()) s += det(a[f[t].a], a[f[t].b], a[f[t].c]);
73 return abs(s) / 6;
74 }
75
76 } // namespace Geo
77 using namespace Geo;

```

7.4 三维向量

```

1 namespace Geo {
2
3 struct p3 {
4     db x, y, z;
5     bool operator==(const p3 &b) const {
6         return !cmp(x, b.x) && !cmp(y, b.y) && !cmp(z, b.z);
7     }
8     bool operator!=(const p3 &b) const { return !(*this == b); }
9     bool operator<(const p3 &b) const {
10         int c = cmp(x, b.x);
11         if(c) return !~c;
12         c = cmp(y, b.y);
13         if(c) return !~c;
14         return !~cmp(z, b.z);
15     }
16 };
17 p3 operator+(p3 a, p3 b) { return {a.x + b.x, a.y + b.y, a.z + b.z}; }
18 p3 operator-(p3 a, p3 b) { return {a.x - b.x, a.y - b.y, a.z - b.z}; }
19 p3 operator~(p3 a) { return {-a.x, -a.y, -a.z}; }
20 p3 operator*(p3 a, db k) { return {a.x * k, a.y * k, a.z * k}; }
21 p3 operator*(db k, p3 a) { return {a.x * k, a.y * k, a.z * k}; }
22 p3 operator/(p3 a, db k) { return {a.x / k, a.y / k, a.z / k}; }
23 p3 operator+=(p3 &a, p3 b) { return a = a + b; }
24 p3 operator-=(p3 &a, p3 b) { return a = a - b; }
25 p3 operator*=(p3 &a, db k) { return a = a * k; }
26 p3 operator/=(p3 &a, db k) { return a = a / k; }
27 db operator*(p3 a, p3 b) { return a.x * b.x + a.y * b.y + a.z * b.z; } // 点积
28 p3 cross(p3 a, p3 b) {
29     return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x};
30 }
31 db det(p3 a, p3 b, p3 c) { return a * cross(b, c); } // 混合积
32 db dis2(p3 a) { return a.x * a.x + a.y * a.y + a.z * a.z; }
33 db dis(p3 a) { return sqrt(dis2(a)); }
34 p3 unit(p3 a) { return a / dis(a); }
35 db cosang(p3 a, p3 b) { return (a * b) / (dis(a) * dis(b)); } // 夹角余弦(要
    求都非零)
36 db sinang(p3 a, p3 b) { return dis(cross(a, b)) / (dis(a) * dis(b)); } // 夹角正弦,恒
    >= 0
37 db angle(p3 a, p3 b) { return atan2(sinang(a, b), cosang(a, b)); } // 夹角,取值
    [0, pi]
38 db area2(p3 a, p3 b, p3 c) { return dis(cross(b - a, c - a)); } // 三角形面积 * 2
39 db volume6(p3 a, p3 b, p3 c, p3 d) { return det(b - a, c - a, d - a); } // 四面体体积
    * 6

```

```

40 p3 perp(p3 a) { // 与 a 垂直的向量(a 非零时结果也非零):取绝对值最小的那个分量来叉
41     return abs(a.x) > abs(a.z) ? p3{a.y, -a.x, 0} : p3{0, -a.z, a.y};
42 }
43 bool colinear(p3 a, p3 b, p3 c) { return cmp(dis(cross(b - a, c - a)), 0) == 0; }
44 bool coplanar(p3 a, p3 b, p3 c, p3 d) { return sign(abs(det(b - a, c - a, d - a)))
    == 0; }
45
46 } // namespace Geo
47 using namespace Geo;

```

7.5 三维平面

```

1 namespace Geo {
2
3 struct plane {
4     p3 n; // 单位法向量
5     db d; //  $n \cdot x = d$ 
6     plane() {}
7     plane(p3 a, p3 b, p3 c) {
8         p3 t = cross(b - a, c - a);
9         db l = sqrt(t * t); // 注意:这里不能写 dis(t) — 会被解析成成员 dis(p3)(遮蔽自由函
    数)
10        if(!sign(l)) {
11            n = {0, 0, 0}, d = 0; // 三点共线:退化(整个平面无意义)
12        } else {
13            n = t / l, d = n * a;
14        }
15    }
16    static plane from(p3 o, p3 nor) {
17        plane r;
18        r.n = unit(nor), r.d = r.n * o;
19        return r;
20    }
21    db side(p3 p) const { return n * p - d; } // 带号距离(n 单位)
22    db dis(p3 p) const { return abs(side(p)); } // 点到平面距离
23    p3 proj(p3 p) const { return p - n * side(p); } // 投影到平面
24    p3 reflect(p3 p) const { return p - n * (2 * side(p)); }
25    bool ons(p3 p) const { return !sign(side(p)); }
26 };
27 bool ispara(const plane &a, const plane &b) { return !sign(sinang(a.n, b.n)); }
28 bool isperp(const plane &a, const plane &b) { return !sign(cosang(a.n, b.n)); }
29 // 两平面夹角:法向量夹角的锐角部分,取值 [0, pi/2]
30 db angle(const plane &a, const plane &b) { return acos(max((db) -1, min((db) 1,
    abs(a.n * b.n)))); }
31 // 两平面的交线:方向  $n1 \times n2$ , 过点  $(d1 \cdot (n2 \times w) + d2 \cdot (w \times n1)) / |w|^2$  ( $w = n1 \times n2$ )
32 // 平行(含重合)时没有唯一的交线:返回方向为零向量的退化直线,调用方用 dis(l.d) == 0 判退化
33 line3 ispl(const plane &a, const plane &b) {
34     p3 w = cross(a.n, b.n);
35     if(!sign(sinang(a.n, b.n))) return line3(a.proj(b.n * b.d), a.proj(b.n * b.d));
36     p3 o = (cross(b.n, w) * a.d + cross(w, a.n) * b.d) / dis2(w);
37     return line3(o, o + w); // 传两点(o, o+w), d 就是 w
38 }

```

```

39 // 直线与平面的交点:解  $n \cdot (o + t \cdot d) = d$  得  $t = (d - n \cdot o) / (n \cdot d)$ 
40 p3 islp(const plane &p, const line3 &l) { return l.o + l.d * ((p.d - p.n * l.o) /
    (p.n * l.d)); }
41 // 过点 o 作平面的垂线
42 line3 perpthrough(const plane &p, p3 o) { return line3(o, o + p.n); }
43 // 直线是否整个在平面内
44 bool ons(const plane &p, const line3 &l) { return !sign(p.side(l.o)) && !sign(p.n *
    l.d); }
45 db dis(const plane &p, p3 x) { return p.dis(x); }
46 p3 proj(const plane &p, p3 x) { return p.proj(x); }
47
48 } // namespace Geo
49 using namespace Geo;

```

7.6 三维旋转

```

1 namespace Geo {
2
3 struct mat3 {
4     db a[3][3];
5     p3 operator*(p3 v) const {
6         return {a[0][0] * v.x + a[0][1] * v.y + a[0][2] * v.z,
7                 a[1][0] * v.x + a[1][1] * v.y + a[1][2] * v.z,
8                 a[2][0] * v.x + a[2][1] * v.y + a[2][2] * v.z};
9     }
10 };
11 mat3 operator*(const mat3 &x, const mat3 &y) {
12     mat3 r{};
13     ForD(i, 0, 3) ForD(j, 0, 3) {
14         db s = 0;
15         ForD(k, 0, 3) s += x.a[i][k] * y.a[k][j];
16         r.a[i][j] = s;
17     }
18     return r;
19 }
20 mat3 transpose(const mat3 &m) {
21     mat3 r{};
22     ForD(i, 0, 3) ForD(j, 0, 3) r.a[i][j] = m.a[j][i];
23     return r;
24 }
25 // 罗德里格斯:  $v' = v \cdot \cos + (k \times v) \cdot \sin + k \cdot (k \cdot v) \cdot (1 - \cos)$ 
26 p3 rot(p3 v, p3 axis, db ang) {
27     p3 k = unit(axis), c = cos(ang), s = sin(ang), d = k * (k * v);
28     return v * c + s * sin(ang) + d * (1 - cos(ang));
29 }
30 p3 rot(p3 v, p3 axis, db ang, p3 o) { return o + rot(v - o, axis, ang); }
31 mat3 rotmat(p3 axis, db ang) {
32     p3 k = unit(axis);
33     db c = cos(ang), s = sin(ang), t = 1 - c;
34     mat3 m{};
35     m.a[0][0] = t * k.x * k.x + c, m.a[0][1] = t * k.x * k.y - s * k.z, m.a[0][2] =
    t * k.x * k.z + s * k.y;

```

```

36     m.a[1][0] = t * k.x * k.y + s * k.z, m.a[1][1] = t * k.y * k.y + c, m.a[1][2] =
    t * k.y * k.z - s * k.x;
37     m.a[2][0] = t * k.x * k.z - s * k.y, m.a[2][1] = t * k.y * k.z + s * k.x, m.a[2]
[2] = t * k.z * k.z + c;
38     return m;
39 }
40 // 以 n 为第三个基向量造右手正交基(u, v, w 两两垂直、都是单位向量、det(u,v,w) = 1)
41 void basis(p3 n, p3 &u, p3 &v, p3 &w) {
42     w = unit(n), u = unit(perp(w)), v = cross(w, u);
43 }
44 p3 tolocal(p3 x, p3 o, p3 u, p3 v, p3 w) {
45     p3 d = x - o;
46     return {u * d, v * d, w * d};
47 }
48 p3 toworld(p3 l, p3 o, p3 u, p3 v, p3 w) { return o + u * l.x + v * l.y + w * l.z; }
49
50 } // namespace Geo
51 using namespace Geo;

```

7.7 三维直线与线段

```

1 namespace Geo {
2
3 struct line3 {
4     p3 o, d; // 参数式 o + t·d
5     line3() {}
6     line3(p3 a, p3 b) : o(a), d(b - a) {}
7     p3 at(db t) const { return o + d * t; }
8 };
9 struct seg3 {
10     p3 x, y;
11     p3 dir() const { return y - x; }
12 };
13
14 // ----- 点与直线 -----
15 p3 proj(const line3 &l, p3 p) { return l.o + l.d * ((l.d * (p - l.o)) /
    dis2(l.d)); }
16 p3 reflect(const line3 &l, p3 p) { return proj(l, p) * 2 - p; }
17 db dis(const line3 &l, p3 p) { return dis(p - proj(l, p)); }
18 bool ons(const line3 &l, p3 p) { return !sign(dis(l, p)); }
19 bool ispara(const line3 &a, const line3 &b) { return !sign(sinang(a.d, b.d)); }
20 bool ispara(const seg3 &a, const seg3 &b) { // 线段方向:某条退化成点时约定为「平行」
    return !sign(dis(a.dir())) || !sign(dis(b.dir())) || !sign(sinang(a.dir(),
    b.dir()));
21 }
22
23 bool isperp(const line3 &a, const line3 &b) { return !sign(cosang(a.d, b.d)); }
24
25 // ----- 两直线:最近点对与距离 -----
26 // pa 在 a 上、pb 在 b 上,dis(pa, pb) 即两直线距离;平行时取 pa = a.o.
27 db dis(const line3 &a, const line3 &b, p3 &pa, p3 &pb) {
28     if(ispara(a, b)) return pa = a.o, pb = proj(b, pa), dis(pa - pb);
29     db A = dis2(a.d), B = a.d * b.d, C = dis2(b.d), den = A * C - B * B;

```

```

30     p3 w = a.o - b.o;
31     db D = a.d * w, E = b.d * w;
32     pa = a.o + a.d * ((B * E - C * D) / den);
33     pb = b.o + b.d * ((A * E - B * D) / den);
34     return dis(pa - pb);
35 }
36 db dis(const line3 &a, const line3 &b) { p3 pa, pb; return dis(a, b, pa, pb); }
37 p3 closest_on(const line3 &a, const line3 &b) { p3 pa, pb; return dis(a, b, pa, pb),
    pa; }
38
39 // ----- 点与线段 -----
40 p3 proj(const seg3 &s, p3 p) {
41     p3 d = s.dir();
42     if(!sign(dis2(d))) return s.x; // 退化成点
43     db t = (d * (p - s.x)) / dis2(d);
44     return s.x + d * max((db) 0, min((db) 1, t));
45 }
46 db dis(const seg3 &s, p3 p) { return dis(proj(s, p) - p); }
47 bool ons(const seg3 &s, p3 p) { return !sign(dis(s, p)); }
48 bool ons_s(const seg3 &s, p3 p) { return ons(s, p) && sign((p - s.x) * (p - s.y)) <
    0; }
49
50 // ----- 两线段:最近点对与距离 -----
51 // pa 在 a 上、pb 在 b 上,返回 |pa - pb|;退化(零长)线段也能用。
52 db dis(const seg3 &a, const seg3 &b, p3 &pa, p3 &pb) {
53     p3 u = a.dir(), v = b.dir();
54     db la = dis(u), lb = dis(v);
55     if(!sign(la) || !sign(lb)) { // 至少一条退化成点
56         if(!sign(la) && !sign(lb)) return pa = a.x, pb = b.x, dis(pa - pb);
57         if(!sign(la)) return pa = a.x, pb = proj(b, pa), dis(pa - pb);
58         return pb = b.x, pa = proj(a, pb), dis(pa - pb);
59     }
60     p3 e = u / la, f = v / lb, w = a.x - b.x;
61     db c = e * f, k = 1 - c * c, p = e * w, q = f * w, s, t;
62     if(ispara(a, b)) { // 平行:最近点对不唯一,s = 0 起步,后面照样投影夹取
63         s = 0, t = q / lb;
64     } else {
65         s = (c * q - p) / (la * k), t = (q - c * p) / (lb * k);
66     }
67     s = max((db) 0, min((db) 1, s)), t = max((db) 0, min((db) 1, t));
68     // |w + s·u - t·v|2 是 (s, t) 上的凸二次函数:盒约束下逐坐标做「一维精确最优 + 夹取」,
69     // 每步都让距离不减,几轮就收敛(平行时的平坦方向也在其中)
70     For(_, 1, 12) {
71         t = max((db) 0, min((db) 1, (q + s * la * c) / lb));
72         s = max((db) 0, min((db) 1, (t * lb * c - p) / la));
73     }
74     pa = a.x + u * s, pb = b.x + v * t;
75     return dis(pa - pb);
76 }
77 db dis(const seg3 &a, const seg3 &b) { p3 pa, pb; return dis(a, b, pa, pb); }
78
79 // 两线段是否相交(公共点存在即真,含端点相接、共线重叠)。
80 bool chkss(const seg3 &a, const seg3 &b) { return !sign(dis(a, b)); }

```

```

81 // 严格相交:交点严格在两段内部(共线/平行/端点相接都不算)。
82 bool chkss_s(const seg3 &a, const seg3 &b) {
83     if(ispara(a, b) || !sign(dis(a.dir())) || !sign(dis(b.dir()))) return false;
84     p3 pa, pb;
85     return !sign(dis(a, b, pa, pb)) && ons_s(a, pa) && ons_s(b, pb);
86 }
87 // 交于唯一一点时输出该点(共线重叠/不相交返回 false)。交点取 a 上的最近点。
88 bool isss(const seg3 &a, const seg3 &b, p3 &p) {
89     if(ispara(a, b) || !sign(dis(a.dir())) || !sign(dis(b.dir()))) return false;
90     p3 pa, pb;
91     return !sign(dis(a, b, pa, pb)) ? (p = pa, true) : false;
92 }
93
94 } // namespace Geo
95 using namespace Geo;

```

7.8 上凸壳

```

1 int upper_hull(int n, p2 *a, p2 *b, bool nos = 0) {
2     if(n <= 0) return 0;
3     sort(a, a + n, [](const p2 &u, const p2 &v) { return u.x != v.x ? u.x < v.x :
    u.y > v.y; });
4     int k = 0;
5     ForD(i, 0, n) {
6         if(k && b[k - 1].x == a[i].x) continue; // 同一个 x(精确相等)只留最高的那个
7         while(k > 1 && crossop({b[k - 2], b[k - 1]}, a[i]) >= (int) nos) --k; //
    nos=0:只留右转;nos=1:共线也留
8         b[k++] = a[i];
9     }
10    return k;
11 }

```

7.9 凸包内点判定

```

1 int in_convex(int n, p2 *a, const p2 &p) {
2     if(n < 3) return 0;
3     if(crossop({a[0], a[1]}, p) < 0 || crossop({a[0], a[n - 1]}, p) > 0) return
    0; // 落在以 a[0] 为心的扇形外
4     if(ons({a[0], a[1]}, p) || ons({a[n - 1], a[0]}, p)) return
    1; // 与 a[0] 相邻的两条边
5     int l = 1, r = n - 1;
6     while(l + 1 < r) { // 找最大的 l,使 p 不在 a[0]->a[l] 的右侧
7         int m = (l + r) >> 1;
8         if(crossop({a[0], a[m]}, p) >= 0)
9             l = m;
10        else
11            r = m;
12    }
13    int c = crossop({a[l], a[l + 1]}, p); // p 只可能在扇形三角形 (a[0],a[l],a[l+1]) 里
14    if(c == 0) return 1; // 落在外边 a[l]a[l+1] 上(含端点 a[l])
15    if(c < 0) return 0; // 落在外边之外(含「对角线超出 a[l]」的情形)

```

```

16     if(crossop({a[0], a[l]}, p) == 0) return ons({a[0], a[l]}, p) ? 2 : 0; // 严格凸
    时对角线在内部
17     return 2;
18 }

```

7.10 半平面交

```

1 vector<p2> hpi(vector<seg> vs) {
2     if(vs.empty()) return {};
3     sort(vs.begin(), vs.end(), [](const seg &a, const seg &b) {
4         db x = a.dir().alpha(), y = b.dir().alpha(); // 按精确极角排序,保证严格弱序
5         if(x != y) return x < y;
6         return crossop(a, b.x) < 0; // 同向时 b 在 a 右侧 -> a 的左侧半平面更小,排前面
7     });
8     vector<seg> ls; // 同向平行(角度差 < eps)只留更强的一条
9     ForD(i, 0, (int) vs.size()) {
10        if(!ls.empty() && !cmp(ls.back().dir().alpha(), vs[i].dir().alpha())) {
11            // 注意:近乎同向的两条直线极角可能只差 1ulp,谁在前是随机的 — 必须显式比「强弱」:
12            // crossop(ls.back(), vs[i].x) >= 0 表示 vs[i] 的限制更紧(或相同),用它替换
13            if(crossop(ls.back(), vs[i].x) >= 0) ls.back() = vs[i];
14            continue;
15        }
16        ls.push_back(vs[i]);
17    }
18    if(ls.size() > 1 && !cmp(ls.front().dir().alpha() - ls.back().dir().alpha() + 2
    * pi, 2 * pi)) {
19        if(crossop(ls.front(), ls.back().x) >= 0)
20            ls.erase(ls.begin());
21        else
22            ls.pop_back();
23    }
24    int n = ls.size();
25    if(n < 3) return {};
26    vector<int> q(n + 1);
27    int h = 0, t = 2; // 队列里是 q[h..t-1],先放前两条
28    q[0] = 0, q[1] = 1;
29    ForD(i, 2, n) {
30        while(t - h >= 2 && crossop(ls[i], isll(ls[q[t - 2]], ls[q[t - 1]])) < 0) --
    t;
31        while(t - h >= 2 && crossop(ls[i], isll(ls[q[h]], ls[q[h + 1]])) < 0) ++h;
32        q[t++] = i;
33    }
34    while(t - h >= 2 && crossop(ls[q[h]], isll(ls[q[t - 2]], ls[q[t - 1]])) < 0) --
    t;
35    while(t - h >= 2 && crossop(ls[q[t - 1]], isll(ls[q[h]], ls[q[h + 1]])) < 0) +
    +h;
36    if(t - h < 3) return {};
37    vector<p2> ret; // 相邻两条直线的交点,按环形顺序:逆时针
38    ForD(i, h, t) ret.push_back(isll(ls[q[i]], ls[q[i + 1 < t ? i + 1 : h]]));
39    vector<p2> qs;
40    ForD(i, 0, (int) ret.size()) if(qs.empty() || !(qs.back() == ret[i]))
    qs.push_back(ret[i]); // 合并重合点
41    while(qs.size() > 1 && qs.front() == qs.back()) qs.pop_back();

```



```

42     if(qs.size() < 3) return {};
43     db s = 0;
44     ForD(i, 0, (int) qs.size()) s += qs[i].det(qs[(i + 1) % qs.size()]);
45     // 交集为空时, 双端队列会退化为一组「自相矛盾」的直线, 交出来的多边形必然是**顺时针**的
46     // (真实的交集多边形一定逆时针: 沿每条直线前进时内部在左)。所以面积 <= 0 一律按空处理 ——
47     // 这一条是「空交集」的唯一判别手段: 不判的话空交集会返回一个既违反输入直线、面积又为负的
48     // 假多边形(实测: 8 条方向大致均匀铺满圆周、偏移 ±1e9 的直线, 真交集为空, 队列却交出了
49     // 一个面积 -1.4e17 的三角形)。这也是教科书式哨兵写法的共同弱点, 这里用 0(k) 的面积符号兜住。
50     if(sign(s) <= 0) return {};
51     return qs;
52 }

```

7.11 圆相关求交与面积交

```

1 struct circle {
2     p2 o;
3     db r;
4 };
5
6 // 有向直线 ab(a -> b) 与 p: p > 0 表示 p 在左侧, < 0 右侧, 0 共线(eps 容差)
7 int side_of(const p2 &a, const p2 &b, const p2 &p) { return sign((b - a).det(p - a)); }
8
9 // 圆与直线的交点。n = 0/1/2 个, 按 t = dir*(p - l.x) 递增排序; 圆心在直线上时关于垂足对称。
10 // 前置: l.x != l.y (退化返回空)。O(1)
11 int cir_line2(const circle &c, const seg &l, p2 *o) {
12     p2 d = l.y - l.x;
13     db L2 = dis2(d);
14     if(!sign(L2)) return 0; // 退化直线
15     p2 f = proj(l, c.o); // 圆心在直线上的投影(垂足)
16     db x = c.r * c.r - dis2(c.o - f); // = (r - |of|)(r + |of|)
17     // 相切判据必须按半径缩放: x ≈ 2r(r - |of|), 除以 r 之后才和 geo 的**绝对** eps = 1e-10 可比。
18     // 不缩放的话半径 1e-6 的圆(判别式 ~1e-12)会被判成相切、只返回 1 个交点。
19     db sc = cmp(c.r, 0) > 0 ? x / c.r : x;
20     if(sign(sc) < 0) return 0; // 相离
21     db h = sign(sc) > 0 ? sqrtl(x) : 0; // 半弦长, 相切时只有垂足一个点
22     p2 u = d / sqrtl(L2); // 直线方向的单位向量
23     int n = 0;
24     if(sign(h) > 0) o[n++] = f + u * h, o[n++] = f - u * h; // 两个交点: 垂足沿直线方向偏移 ±h
25     else o[n++] = f; // 相切: 1 个点
26     return n;
27 }
28 vector<p2> cir_line(const circle &c, const seg &l) {
29     p2 t[2];
30     int n = cir_line2(c, l, t);
31     vector<p2> r(t, t + n);
32     if(n == 2 && t[0] * (l.dir()) > t[1] * (l.dir())) swap(r[0], r[1]); // 按参数(取 l.x 为原点)递增

```

```

33     return r;
34 }
35
36 // 圆与线段的交点(直线交点再用 ons 过滤)。按 t 递增。O(1)
37 vector<p2> cir_seg(const circle &c, const seg &l) {
38     vector<p2> r;
39     for(p2 x : cir_line(c, l))
40         if(ons(l, x)) r.push_back(x);
41     return r;
42 }
43
44 // 两圆的交点: 0(外离/内含/内离/同心/重合)/ 1(内切或外切)/ 2 个, 按相对 c1.o 的极角排序。
45 // 做法: 沿 o1->o2 方向向量 d, 再沿垂线取 ±h, 并逐个验证到两圆心的距离确实都是 r。O(1)
46 vector<p2> cir_cir(const circle &c1, const circle &c2) {
47     p2 dv = c2.o - c1.o;
48     db d = dis(dv);
49     if(!sign(d)) return {}; // 同心/重合: 无孤立交点
50     p2 u = dv / d; // o1 -> o2 的单位方向
51     db x = (d * d + c1.r * c1.r - c2.r * c2.r) / (2 * d); // 交点连线到 o1 的距离
52     db h2 = c1.r * c1.r - x * x;
53     // 同 cir_line2: 相切判据按半径缩放(否则半径 1e-6 的两圆会被判成相切/相离)
54     db sc2 = cmp(c1.r, 0) > 0 ? h2 / c1.r : h2;
55     if(sign(sc2) < 0) return {}; // 两圆相离(含内含)
56     vector<p2> r;
57     p2 base = c1.o + u * x;
58     if(sign(sc2) > 0) {
59         db h = sqrtl(h2);
60         p2 w = r90(u) * h;
61         r.push_back(base + w), r.push_back(base - w);
62     } else
63         r.push_back(base); // 相切: 两处解重合, 只留 1 个
64     auto good = [&](const p2 &p) {
65         db t1 = dis(p - c1.o) - c1.r, t2 = dis(p - c2.o) - c2.r;
66         db e = eps * (1 + max(c1.r, c2.r));
67         return abs(t1) <= e && abs(t2) <= e; // 防止 r = 0 / 极端半径
68     };
69     vector<p2> q;
70     for(p2 p : r)
71         if(good(p)) q.push_back(p);
72     sort(q.begin(), q.end(), [&](const p2 &a, const p2 &b) { return (a - c1.o).alpha() < (b - c1.o).alpha(); });
73     return q;
74 }
75
76 // 两圆「交」面积(解析法: 两个弓形之和)。设 d 为圆心距(0 < |r1 - r2| < d < r1 + r2),
77 // x = (d^2 + r1^2 - r2^2) / (2d) 是公共弦到圆 1 圆心的距离, h = sqrt(r1^2 - x^2) 是公共弦的一半,
78 // a_i = 2 * acos(...) 是交点在圆 i 里的圆心角; 则 S = (r1^2 * a1 / 2 - d * h / 2) + (r2^2 * a2 / 2 - d * h / 2),
79 // 即两个「扇形 - 三角形」弓形之和。内含/内切/重合返回 pi * min(r1, r2)^2 (小圆整个被盖住),
80 // 外离/外切返回 0(容差 eps)。形参无半径大小要求, 内部按 r 归一。O(1)

```

```

81 db cir_cir_area(const circle &c1, const circle &c2) {
82     db r1 = c1.r, r2 = c2.r;
83     if(r1 < r2) swap(r1, r2); // 保证 r1 >= r2
84     db d = dis(c1.o - c2.o);
85     if(d >= r1 + r2 - eps) return 0; // 外离 / 外切:交面积 0
86     if(d <= r1 - r2 + eps) return pi * r2 * r2; // 内含 / 内切:小圆整个被
    盖住
87     db a1 = 2 * acosl((r1 * r1 + d * d - r2 * r2) / (2 * r1 * d)); // 交点在圆 1 里
    的圆心角
88     db a2 = 2 * acosl((r2 * r2 + d * d - r1 * r1) / (2 * r2 * d)); // 交点在圆 2 里
    的圆心角
89     db x = (d * d + r1 * r1 - r2 * r2) / (2 * d); // 公共弦到圆 1 圆心的距
    离
90     db h = sqrtl(max((db) 0, r1 * r1 - x * x)); // 公共弦的一半
91     return max((db) 0, (r1 * r1 * a1 / 2 - d * h / 2) + (r2 * r2 * a2 / 2 - d * h /
    2));
92 }
93
94 // — cir_poly_area 的零件:圆盘与「三角形(圆心, a, b)」的交(有向面积 * 2) —
95 // 从 u 到 v 的**有向**夹角,取值 (-pi, pi]:atan2(叉积, 点积)。
96 // 这里必须用有向角,不能把角差归一到 [0, 2pi) — 圆心落在多边形外时各边扇形角的有向和是 0,
97 // 归一化之后会变成 2pi,于是「圆完全在多边形外」会算出整整一块圆面积(实测过的 bug)。
98 db arg2(const p2 &u, const p2 &v) { return atan2l(u.det(v), u * v); }
99 // a、b 先平移到「以圆心为原点」;va/vb = a、b 是否在圆盘内(含边界)。三种情形:
100 // · a、b 都在圆内 -> 整块三角形都在圆内:a.det(b)
101 // · 线段 ab 与圆盘无交 -> 只贡献一个扇形:arg2(a, b) * r^2
102 // · 否则 -> 沿 a -> b 把「弧 + 弦上那段直线段」依次拼起来(扇形 + 三角形)
103 db tri_disk2(const circle &c, p2 a, p2 b) {
104     a = a - c.o, b = b - c.o;
105     db r2 = c.r * c.r;
106     // 判「点是否在圆盘内」比的是**距离**而不是平方距离:平方距离在半径 1e-6 时只有 1e-12,
107     // 会被 geo 的绝对 eps = 1e-10 全部判成「在圆内」
108     int va = cmp(dis(a), c.r) <= 0, vb = cmp(dis(b), c.r) <= 0;
109     if(va && vb) return a.det(b);
110     vector<p2> v = cir_seg(circle{{0, 0}, c.r}, {a, b}); // 圆(圆心在原点)与线段 ab 的
    交点,按 a->b 递增
111     if(v.empty()) return arg2(a, b) * r2; // 整条边在圆盘外:纯扇形(有向
    角,不用归一)
112     db s = 0;
113     s += va ? a.det(v[0]) : arg2(a, v[0]) * r2; // 起点那一侧的扇形/三角形
114     s += vb ? v.back().det(b) : arg2(v.back(), b) * r2; // 终点那一侧
115     if(v.size() > 1) s += v[0].det(v[1]); // 圆内那段弦与圆心围成的三角
    形
116     return s;
117 }
118
119 // 逆时针**简单**多边形 a[0..n-1](凹也对)与圆的交面积,恒 >= 0。0(n)
120 // 做法:对每条边 (a[i], a[i+1]) 求「三角形(圆心, a[i], a[i+1]) 与圆盘」的有向交面积并累加 —
121 // 这些三角形是圆心的「扇形剖分」,有向面积自动抵消外部与凹口,所以凹多边形同样正确。
122 // 退化:r 在 eps 内按 0 处理;多边形面积 0 时结果 0。顺时针多边形不在契约内(会返回 0)。
123 db cir_poly_area(int n, p2 *a, const circle &c) {
124     if(n < 3 || !sign(c.r)) return 0;
125     db s = 0;

```

```

126     ForD(i, 0, n) s += tri_disk2(c, a[i], a[(i + 1) % n]);
127     return s > 0 ? s / 2 : 0; // 逆时针 -> 有向和为正;数值噪声造成的极小负值归零
128 }

```

7.12 多边形包含

```

1 int wn_contain(int n, p2 *a, const p2 &p) {
2     if(n < 3) return 0;
3     int wn = 0;
4     ForD(i, 0, n) {
5         p2 u = a[i], v = a[(i + 1) % n];
6         if(u == v) continue; // 退化边(相邻重复点):跳过,否则 ons 会认为任意点都在它上面
7         if(ons({u, v}, p)) return 1;
8         int su = sign(u.y - p.y), sv = sign(v.y - p.y);
9         if(su <= 0 && sv > 0 && crossop({u, v}, p) > 0) ++wn; // 向上穿过 p 所在
    水平线且 p 在左
10        else if(su > 0 && sv <= 0 && crossop({u, v}, p) < 0) --wn; // 向下穿过且 p 在
    右
11    }
12    return wn ? 2 : 0;
13 }

```

7.13 最小圆覆盖

```

1 struct circle {
2     p2 o;
3     db r;
4 };
5 circle cir(p2 a, p2 b) { return {(a + b) / 2, dis(a - b) / 2}; } // 以 ab 为直径的圆
6 bool in_circle(const circle &c, const p2 &x) { return cmp(dis(c.o - x), c.r) <= 0; }
7 circle mcc(vector<p2> a) {
8     if(a.empty()) return {{0, 0}, -1};
9     static mt19937_64
    mcc_rng(chrono::steady_clock::now().time_since_epoch().count());
10    shuffle(a.begin(), a.end(), mcc_rng);
11    circle c = {a[0], 0};
12    ForD(i, 1, (int) a.size()) {
13        if(in_circle(c, a[i])) continue;
14        c = cir(a[0], a[i]);
15        ForD(j, 0, i) {
16            if(in_circle(c, a[j])) continue;
17            c = cir(a[i], a[j]);
18            ForD(k, 0, j) {
19                if(in_circle(c, a[k])) continue;
20                if(!crossop({a[i], a[j]}, a[k])) { // 三点共线:最小圆取最远两点的直径
21                    db dij = dis2(a[i] - a[j]), dik = dis2(a[i] - a[k]), dj =
    dis2(a[j] - a[k]);
22                    if(dij >= dik && dij >= dj)
23                        c = cir(a[i], a[j]);
24                    else if(dik >= dj)
25                        c = cir(a[i], a[k]);
26                    else

```



```

27         c = cir(a[j], a[k]);
28     } else {
29         auto o = circumcenter(a[i], a[j], a[k]);
30         p2 oc = {o[0], o[1]};
31         c = {oc, dis(oc - a[k])};
32     }
33 }
34 }
35 }
36 return c;
37 }

```

7.14 最近点对

```

1 db dis_closest(vector<p2> a) {
2     int n = a.size();
3     if(n < 2) return 1e100L;
4     sort(a.begin(), a.end(), [](const p2 &u, const p2 &v) { return u.x != v.x ? u.x
< v.x : u.y < v.y; });
5     vector<p2> b(n); // 归并缓冲 + 窄条缓冲,只分配一次
6     db ans = 1e100L;
7     auto rec = [&](auto &&self, int l, int r) -> void { // 处理 [l, r),结束后 a[l..r)
按 y 有序
8         if(r - l <= 1) return;
9         int mid = (l + r) >> 1;
10        db mx = a[mid].x;
11        self(self, l, mid), self(self, mid, r);
12        int i = l, j = mid, k = l; // 归并两半(都按 y 有序)
13        while(i < mid || j < r) b[k++] = (j >= r || (i < mid && a[i].y <= a[j].y)) ?
a[i++] : a[j++];
14        ForD(t, l, r) a[t] = b[t];
15        int m = 0; // 只留下横坐标距分界线 < ans 的点(按 y 递增)
16        ForD(t, l, r)
17            if(abs(a[t].x - mx) < ans) b[m++] = a[t];
18        ForD(t, 0, m) for(int u = t - 1; u >= 0 && b[t].y - b[u].y < ans; --u) ans =
min(ans, dis(b[t] - b[u]));
19    };
20    rec(rec, 0, n);
21    return ans;
22 }

```

7.15 简单多边形三角剖分

```

1 vector<array<int, 3>> ear_clip(int n, p2 *a) {
2     vector<array<int, 3>> ret;
3     if(n < 3) return ret;
4     // [p 严格落在逆时针三角形 (x, y, z) 内部] : 三条边的左半平面都在左侧
5     auto inside = [&](const p2 &p, const p2 &x, const p2 &y, const p2 &z) {
6         return (y - x).det(p - x) > 0 && (z - y).det(p - y) > 0 && (x - z).det(p -
z) > 0;
7     };
8     // 环上的前驱、后继: 用下标数组维护双向链表, 不真的删点, 避免下标错位

```

```

9     // cnt 是环上剩下的点数, 不能拿 n 当计数器 — n 还是数组的大小
10    vector<int> prv(n), nxt(n);
11    ForD(i, 0, n) prv[i] = (i + n - 1) % n, nxt[i] = (i + 1) % n; // ForD(i, l, r) 即
i = l..r-1
12    int cnt = n;
13    auto live = [&](int x) { return prv[nxt[x]] == x && nxt[prv[x]] == x; }; // 还在
环上
14    auto drop = [&](int x) {
15        int p = prv[x], q = nxt[x];
16        nxt[p] = q, prv[q] = p;
17        --cnt;
18    };
19    // 预处理: 摘掉三点共线的中间点。判据用「局部相对」容差: 三个点的坐标本来就带舍入,
20    // 插在边上的点常常只是  $\det \approx 1e-14$  而非精确 0, 若只用 == 0 会漏掉它们, 之后就会切出
21    // 一个面积  $\sim 1e-14$  的退化三角形。容差取  $1e-13 * |cur - prev| * |next - cur|$  (叉积的
22    // 浮点误差上界就是这个量级), 这样薄三角形(底  $2e9$  高 1,  $\det = 2e9$ )不会被误判成共线。
23    // 只有这一步用容差, 耳切本身仍然用精确比较。
24    auto colin = [&](int x) {
25        p2 u = a[x] - a[prv[x]], v = a[nxt[x]] - a[x];
26        return abs(u.det(v)) <= 1e-13L * dis(u) * dis(v);
27    };
28    // 摘掉一个点会暴露新的共线点, 所以反复扫, 直到环上不再有共线点
29    for(bool ch = 1; ch && cnt >= 3; ) {
30        ch = 0;
31        for(int x = 0; x < n; ++x)
32            if(live(x) && cnt > 3 && colin(x)) drop(x), ch = 1;
33    }
34    if(cnt < 3) return ret; // 全是共线点: 面积 0, 没有合法三角形
35    for(; cnt > 2; ) {
36        int ear = -1;
37        for(int cur = 0; cur < n; ++cur) {
38            if(!live(cur)) continue;
39            int p = prv[cur], q = nxt[cur];
40            if(p == cur || q == cur || p == q) continue;
41            if((a[cur] - a[p]).det(a[q] - a[cur]) <= 0) continue; // 该顶点不是严格凸
的
42            bool ok = 1;
43            for(int i = nxt[q]; i != p && ok; i = nxt[i])
44                if(inside(a[i], a[p], a[cur], a[q])) ok = 0;
45            if(ok) {
46                ear = cur;
47                break;
48            }
49        }
50        if(ear < 0) break; // 一个耳都没有(输入不满足前置条件): 剩下的切不动, 直接停
51        int p = prv[ear], q = nxt[ear];
52        ret.push_back({p, ear, q}); // (prev, cur, next) 左转, 按逆时针输出
53        drop(ear);
54    }
55    // 只剩 3 个点时上面那个循环不再进入: 若这三点共线(整个多边形面积 0), 没有合法三角形
56    if(cnt == 3 && (a[nxt[0]] - a[0]).det(a[prv[0]] - a[0]) == 0) ret.clear();
57    return ret;
58 }

```

```

59 // 注：若确实想用 geo.cpp 的 sign 做容差判定，请先把 det 归一化 — sign 用的是绝对 eps，
60 //    大坐标下几乎不生效(1e18 的 det 永远非 0)，小尺度下又会把整个多边形判成退化。

```

7.16 多边形重心

```

1 db perimeter(int n, p2 *a) {
2     db s = 0;
3     ForD(i, 0, n) s += dis(a[(i + 1) % n] - a[i]);
4     return s;
5 }
6 p2 centroid(int n, p2 *a) {
7     if(n == 0) return {0, 0};
8     p2 ret = {0, 0};
9     db s = 0;
10    ForD(i, 0, n) {
11        p2 u = a[i], v = a[(i + 1) % n];
12        db c = u.det(v);
13        s += c;
14        ret = ret + (u + v) * c;
15    }
16    if(!sign(s)) { // 退化: 退回顶点平均
17        p2 av = {0, 0};
18        ForD(i, 0, n) av = av + a[i];
19        return av / (db) n;
20    }
21    return ret / (3 * s);
22 }

```